

Министерство образования Республики Беларусь
Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра программного обеспечения информационных технологий

Дисциплина: Базы данных (БД)

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовому проекту
на тему:

«База данных фитнес-центра»

БГУИР КП 1-40 01 01 022 ПЗ

Студент
Руководитель

Наривончик А.М.
Марина И.М.

Минск 2026

СОДЕРЖАНИЕ

Введение.....	5
1 Анализ предметной области	6
1.1 Анализ существующих аналогов	6
1.1.1 Система фитнес-клубов «Адреналин».....	6
1.1.2 Программа «Абонемент»	7
1.1.3 Платформа «Gulliver CRM».....	9
1.2 Постановка задачи.....	10
1.2.1 Назначение разработки	10
1.2.2 Перечень функциональных требований.....	10
1.2.3 Входные данные.....	11
1.2.4 Выходные данные	12
2 Анализ и разработка функциональных требований	14
2.1 Используемые технологии при разработке	14
2.2 Функциональные требования к базе данных	14
2.3 Ключевые особенности разрабатываемой базы данных.....	16
2.4 Алгоритм обработки данных и методы взаимодействия.....	16
3 Инфологическая модель предметной области	17
3.1 Модель базы данных.....	17
3.2 Словесное описание модели	17
4 Подробное описание бизнес-логики	20
4.1 Хранимые процедуры и функции.....	20
4.1.1 Вычисление итоговой стоимости платежной позиции	20
4.1.2 Вычисление общей суммы платежа.....	21
4.1.3 Оформление посещения клиента	21
4.1.4 Добавление новой услуги фитнес-центра	23
4.1.5 Обновление тренерского опыта работы	24
4.1.6 Заполнение графика тренера на день.....	24
4.2 Триггеры	28
4.2.1 Проверка отмены записи не позднее чем за час до занятия.....	28
4.2.2 Продление срока абонемента при создании заморозки.....	28
4.2.3 Освобождение места при отмене бронирования	29
4.2.4 Запрет записи клиента на занятие без активного абонемента	29
4.3 Представления	31
4.3.1 Отчёт по загрузке залов.....	31
4.3.2 Отчёт по популярности видов занятий.....	32
4.3.3 Финансовая сводка	32
4.3.4 Отчёт по эффективности тренеров	32
4.3.5 Отчёт по «спящим» клиентам	33
5 Тестирование базы данных	34
Заключение	36
Приложение А	40

ВВЕДЕНИЕ

Фитнес-индустрия в последние годы переживает стремительный рост: количество посетителей фитнес-центров увеличивается, расширяется спектр услуг, а клиенты становятся всё более требовательными к качеству сервиса. Современный фитнес-центр – это не просто зал с тренажёрами, а сложная экосистема, включающая управление абонементом, расписанием занятий, работой тренеров, учётом посещений, продажей дополнительных услуг (солярий, массаж, бар) и системой лояльности.

Разработка специализированной базы данных фитнес-центра позволяет автоматизировать эти процессы: вести учёт клиентов и их абонементов, контролировать загрузку залов и тренеров, формировать отчёты о посещаемости и выручке, отправлять клиентам напоминания о предстоящих тренировках. Внедрение такой системы повышает прозрачность работы клуба, снижает нагрузку на администраторов и создаёт дополнительную ценность для клиента через удобство записи и прозрачность истории его занятий.

Предметная область «фитнес-центр» предъявляет к базе данных и программному обеспечению ряд жёстких требований, которые определяют фокус разработки. Во-первых, это производительность и многопользовательский доступ: в часы пик система одновременно используется десятками администраторов, тренеров и сотнями клиентов. Во-вторых, целостность и непротиворечивость данных критически важна: нельзя допустить, чтобы два клиента записались к одному тренеру в одно и то же время, или чтобы количество оставшихся посещений по абонементу стало отрицательным. В-третьих, гибкость архитектуры необходима для поддержки разных типов абонементов, расписания занятий и разнообразных ценовых пакетов. В рамках данной курсовой работы основное внимание направлено на реализацию именно этих аспектов: проектирование нормализованной реляционной базы данных, включающей минимум 40 таблиц, охватывающих справочники (клиенты, тренеры, типы абонементов, виды занятий), и оперативные сущности (записи на занятия, продажа абонементов, посещения, оплаты). Важным аспектом также является реализация триггеров для автоматического контроля остатка посещений и предотвращения двойной записи, хранимых процедур для формирования расписания на неделю и расчёта зарплаты тренеров, а также представлений для административной отчётности по загрузке залов. Разработанная база данных полностью покрывает ключевые бизнес-процессы фитнес-центра и может быть использована в качестве основы для реальной автоматизации малого или среднего клуба.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

Современный рынок фитнес-услуг Беларуси характеризуется высокой конкуренцией и ростом требований клиентов к качеству сервиса. Согласно отраслевым исследованиям, многие белорусские фитнес-клубы сталкиваются с проблемой низкого уровня автоматизации. По оценкам экспертов, около 80% рынка не автоматизированы комплексно, а имеющиеся решения часто внедряются хаотично, без интеграции друг с другом, что приводит к необходимости ручного переноса данных между системами. Это создает предпосылки для разработки специализированных баз данных, способных обеспечить централизованное управление всеми бизнес-процессами.

1.1 Анализ существующих аналогов

В ходе работы были проанализированы три системы, применяющиеся в белорусских фитнес-центрах и описанные в научной литературе.

1.1.1 Система фитнес-клубов «Адреналин»

Данное решение, представленное корпоративным сайтом и личным кабинетом клиента, автоматизирует взаимодействие с посетителями: информирование о расписании, онлайн-запись на групповые программы и отслеживание баланса посещений. Интерфейс системы представлен на рисунке 1.1.1.

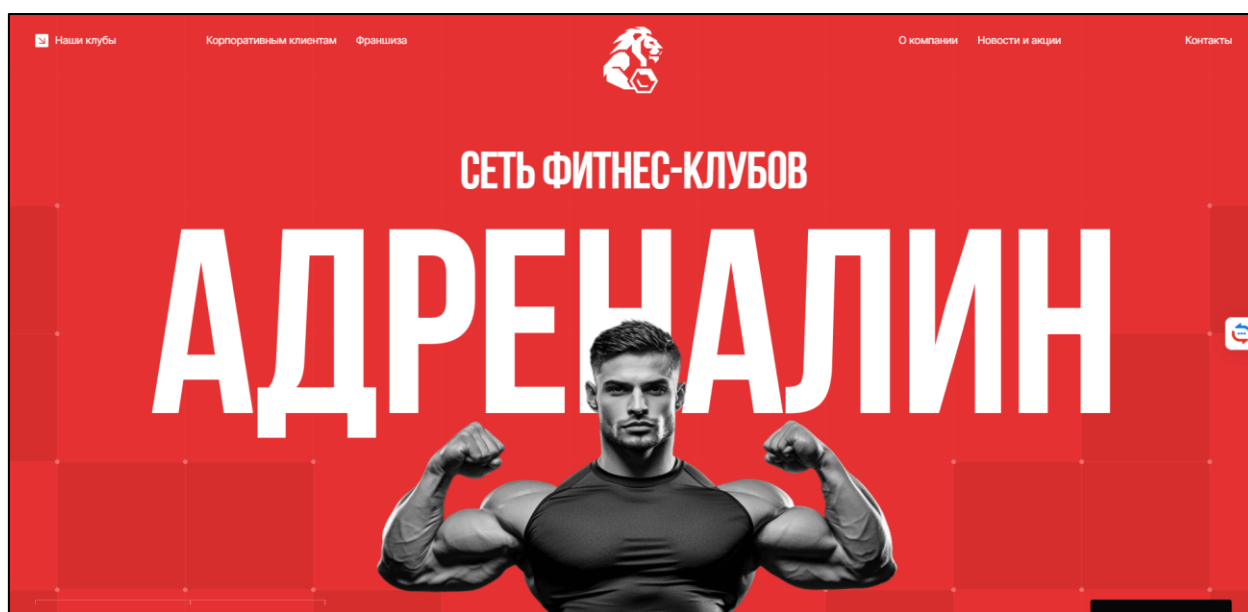


Рисунок 1.1.1 – Интерфейс системы фитнес-клубов «Адреналин»

Достоинства:

- Централизация услуг: пользователь получает доступ к единой точке входа для всех клубов сети, что позволяет выбирать удобное место для тренировки без повторной регистрации.

- Интеграция с клубной системой: синхронизация расписания в реальном времени снижает риск двойной записи и упрощает процесс бронирования мест в популярных группах.

- Информационная поддержка: наличие новостной ленты и актуального предложения услуг (цены, акции) повышает лояльность клиентов за счет прозрачности.

Недостатки:

- Нет функции «заморозки» абонемента онлайн. Чтобы приостановить абонемент (например, из-за болезни или отпуска), нужно писать заявление в клубе или менеджеру – это занимает время и требует личного присутствия.

- Нет истории посещений и чекинов. Пользователь не может посмотреть, когда пришёл и ушёл из клуба, сколько тренировок осталось по абонементу, когда он истекает.

- Нет возможности купить абонемент онлайн с оплатой картой прямо на сайте. Обычно это просто форма «оставьте заявку», после которой перезванивает менеджер.

С точки зрения архитектуры, сайт опирается на защищенное HTTPS-соединение и стандартные протоколы передачи данных, содержит так же адаптивный дизайн для разных устройств.

1.1.2 Программа «Абонемент»

Данное решение от компании UCS автоматизирует ключевые процессы: регистрацию посетителей через турникеты, запись на тренировки, учет рабочего времени специалистов. Интерфейс системы представлен на рисунке 1.1.2.

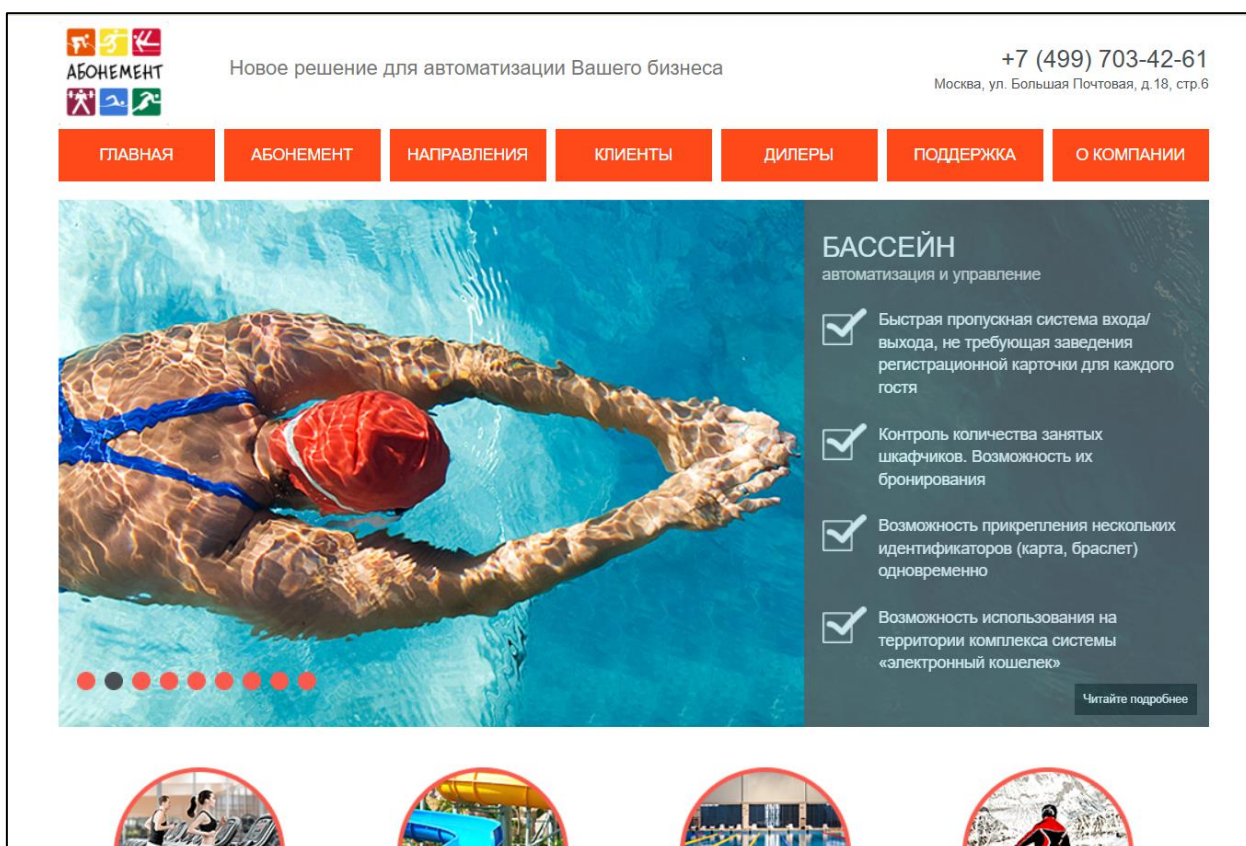


Рисунок 2.1.2 – Интерфейс системы «Абонемент»

Достоинства:

- Интеграция с турникетами на уровне контроллеров и считывателей, что обеспечивает точный контроль доступа.
- Развитая отчетность: доступны отчеты по загрузке залов и тренеров, прибыльности услуг, выявлению лояльных клиентов .
- Формирование финансовой отчетности и аналитика по всем сегментам бизнеса.

Недостатки:

- Отсутствует мобильная версия приложения для клиентов с самостоятельной записью и историей посещений.
- Не указана возможность гибкой настройки различных типов абонементов (с «заморозкой», семейных) под изменяющиеся маркетинговые потребности.

С точки зрения базы данных, платформа опирается на высоконагруженные распределённые системы. Однако внутренняя схема закрыта. Преимущества: масштабируемость, кэширование, гибкая система скидок.

1.1.3 Платформа «Gulliver CRM»

Это облачная CRM-система, ориентированная на управление взаимоотношениями с клиентами. В клубе CROSSiT система используется для управления групповыми занятиями. Интерфейс системы представлен на рисунке 1.1.3.

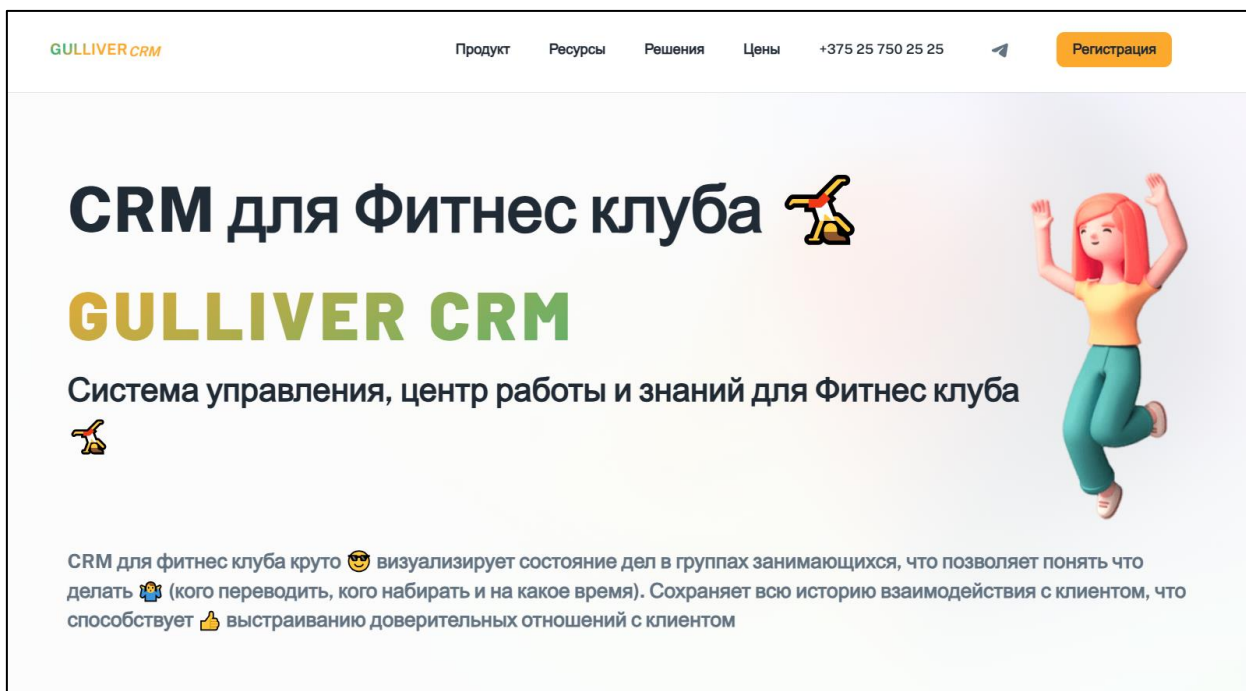


Рисунок 1.1.3. – Интерфейс «Gulliver CRM»

Достоинства:

- Интуитивная визуализация наполняемости групп (цветовая индикация), позволяющая быстро выявлять проблемные места в расписании.
- Мощный биллинг (выставление счетов и взаиморасчеты), способный обрабатывать сложные случаи: частичные оплаты, возвраты, переводы платежей между клиентами, разницу в цене групповых и индивидуальных занятий.

Недостатки:

- Система ориентирована в первую очередь на групповые программы и может быть избыточно сложна или, наоборот, не обладать специфическим функционалом для классических тренажерных залов.

Проведенный анализ показывает, что существующие на белорусском рынке решения либо являются дорогими монолитными комплексами (например, «Абонемент»), либо сфокусированы на отдельных задачах (CRM для продаж). Проектируемая база данных направлена на «гибридный» сегмент рынка: небольшие и средние клубы, а также персональные тренинг-студии, где внедрение тяжелых ERP-систем экономически нецелесообразно.

1.2 Постановка задачи

1.2.1 Назначение разработки

В рамках данной курсовой работы планируется разработать сложную реляционную базу данных для программного продукта – автоматизированной системы управления фитнес-центром с расширенным функционалом учёта абонементов, записи на занятия, управления тренерами и контроля посещаемости.

Для выполнения поставленной задачи необходимо спроектировать физическую модель базы данных для реляционной СУБД (MySQL) с соблюдением третьей нормальной формы, обеспечив целостность данных через систему внешних ключей и ограничений; реализовать расширенную бизнес-логику с помощью хранимых процедур, пользовательских функций и триггеров для автоматизации ключевых процессов – автоматическое списание посещений по абонементу при входе клиента, проверка доступности мест при записи на групповое занятие, расчёт заработной платы тренера на основе проведённых тренировок, контроль срока действия абонемента и автоматическое обновление его статуса («активен», «заморожен», «истёк»); создать систему аналитических представлений для удобного получения отчётной информации: финансовая сводка по продаже абонементов и услуг, загруженность тренеров и залов, популярные виды занятий, список клиентов с истекающими абонементами, аналитика по оттоку клиентов; а также протестировать разработанную базу данных на предмет корректности структуры, ограничений целостности и отработки триггеров и хранимых процедур.

1.2.2 Перечень функциональных требований

Система должна реализовывать следующие основные функции на уровне БД:

- Хранение информации о клиентах: полное имя, контактный телефон, электронная почта, дата регистрации в клубе, статус (активен, заблокирован).
- Поддержка различных типов абонементов: лимитированные (фиксированное число посещений), безлимитные (на 1, 3, 6 или 12 месяцев), абонементы с возможностью заморозки.
- Фиксация продажи абонементов: дата начала и окончания действия, количество оплаченных посещений (для лимитированных), сумма покупки, статус (активен, заморожен, истёк).
- Автоматический контроль остатка посещений по лимитированному абонементу с запретом записи при исчерпании лимита.
- Ведение истории посещений клиента с фиксацией даты, времени, типа занятия.
- Хранение информации о видах занятий: название, продолжительность в минутах, максимальная вместимость группы.

- Поддержка расписания занятий с привязкой к конкретному залу, тренеру, дате и времени начала/окончания.
- Реализация записи клиентов на занятия: фиксация даты и времени записи, статуса записи (подтверждена, отменена, проведена).
- Автоматическая проверка доступности мест при записи: запрет записи, если количество уже записанных клиентов превышает максимальную вместимость зала.
- Хранение информации о тренерах: полное имя, специализация, контактный телефон, стаж работы, статус (работает, в отпуске, уволен).
- Привязка тренеров к конкретным занятиям в расписании.
- Учёт проведённых тренером занятий с фиксацией даты, времени, типа занятия и количества клиентов.
- Расчёт заработной платы тренера на основе проведённых занятий (фиксированная ставка за занятие).
- Хранение каталога дополнительных услуг: название (солярий, массаж, сауна, спортивный бар), цена.
- Формирование отчёта по загрузке залов: средняя наполняемость в процентах за выбранный период.
- Формирование отчёта по популярности видов занятий: количество записей, количество проведённых занятий.
- Формирование финансовой сводки: общая выручка, выручка по типам услуг, динамика продажи абонементов по месяцам.

1.2.3 Входные данные

В процессе проектирования требований были определены следующие входные данные для системы:

1. Данные о клиентах:

- полное имя (фамилия, имя, отчество при наличии);
- контактный телефон;
- адрес электронной почты;
- дата регистрации в клубе.

2. Данные об абонементных:

- тип абонемента (лимитированный / безлимитный);
- количество посещений (для лимитированных абонементов);
- срок действия (1, 3, 6 или 12 месяцев для безлимитных);
- стоимость абонемента;
- дата начала действия;
- статус заморозки с указанием периода заморозки.

3. Данные о занятиях и расписании:

- название вида занятия (йога, пилатес, кроссфит);
- продолжительность занятия в минутах;
- максимальная вместимость группы;
- привязка к конкретному залу;
- привязка к тренеру;

- дата и время начала / окончания занятия.
- 4. Данные о записи на занятия:
 - идентификатор клиента;
 - идентификатор занятия;
 - дата и время записи (фиксируется автоматически);
 - статус записи (подтверждена, отменена, проведена).
- 5. Данные о тренерах:
 - полное имя тренера;
 - специализация (силовой тренинг, йога, единоборства);
 - контактный телефон;
 - стаж работы (в годах);
 - фиксированная ставка за одно занятие;
 - статус (работает, в отпуске, уволен).
- 6. Данные о дополнительных услугах:
 - название услуги (солярий, массаж, сауна, спортивный бар);
 - цена услуги;
 - идентификатор клиента, заказавшего услугу;
 - дата и время оказания услуги.
- 7. Финансовые данные:
 - сумма платежа;
 - способ оплаты (наличные, банковская карта, онлайн-перевод);
 - дата и время платежа.
- 8. Данные о посещениях:
 - идентификатор клиента;
 - идентификатор списанного абонеента (для лимитированных);
 - дата и время входа в фитнес-центр (фиксируется автоматически);
 - тип проведённого занятия.

1.2.4 Выходные данные

Выходными данными разрабатываемой базы данных являются:

- результаты выполнения SQL-запросов (выборки клиентов по статусу абонеента, записей на занятия по клиенту или по дате, списка проведённых занятий по тренеру, истории посещений конкретного клиента, перечня активных абонеентов с истекающим сроком действия);
- отчёт по загрузке залов (средняя наполняемость в процентах за выбранный период);
- отчёт по популярности видов занятий (количество записей, количество проведённых занятий);
- финансовая сводка (общая выручка, выручка по типам услуг, динамика продажи абонеентов по месяцам);
- статистика по тренерам (количество проведённых занятий, общее количество клиентов на занятиях, начисленная заработная плата за период).

1.2.5 Требования к составу и параметрам технических средств

БД и обслуживающее её серверное ПО должны функционировать на серверах или персональных компьютерах со следующими минимальными характеристиками:

- СУБД: MySQL версии 8.0 и выше;
- процессор: Intel Core i5 / AMD Ryzen 5 или серверный аналог;
- оперативная память: от 4 GB;
- накопитель: SSD объемом от 100 GB (для быстрого чтения индексов, истории заказов, геоданных и журналов событий);
- операционная система: предпочтительно Linux (Ubuntu/CentOS) для production, поддержка Windows и macOS для среды разработки.

1.2.6 Требования к информационной и программной совместимости

Система должна обеспечивать:

- возможность подключения backend-приложений посредством стандартных драйверов;
- строгое соблюдение стандартов ANSI SQL там, где это возможно, с минимизацией использования нестандартных расширений для обеспечения потенциальной миграции на другие реляционные СУБД;
- корректную обработку кодировки UTF-8 для хранения имён клиентов на разных языках, спецсимволов в названиях тренировок.

2 АНАЛИЗ И РАЗРАБОТКА ФУНКЦИОНАЛЬНЫХ ТРЕБОВАНИЙ

2.1 Используемые технологии при разработке

В данном проекте в качестве основной СУБД выбрана MySQL 8.4. Эта реляционная база данных предоставляет отличный баланс между производительностью, надёжностью и простотой администрирования. В контексте фитнес-центра выбор реляционной модели обоснован высокой степенью связности данных: клиенты связаны с приобретёнными абонементом и историей посещений, абонементы – с занятиями и списаниями, занятия – с расписанием, тренерами и залами, а также с записями клиентов и платежами. MySQL отлично справляется с JOIN-запросами на больших объёмах данных при правильном проектировании индексов.

Основные возможности MySQL 8.4, используемые в проекте:

- Движок InnoDB – обеспечивает поддержку ACID-транзакций, что критически важно для финансового модуля (учёт платежей за абонементы и дополнительные услуги), управления записью на занятия (невозможность двойной записи одного клиента на одно занятие при сбое).

- Хранение связей "многие-ко-многим" – реализуется через кросс-таблицы (например, `client_abonements` для связи клиентов и приобретённых абонементов, `abonement_visits` для связи абонементов и списаний посещений, `trainer_schedule` для связи тренеров и занятий в расписании).

- Триггеры и хранимые процедуры – используются для выполнения сложной бизнес-логики на стороне базы данных (автоматический контроль остатка посещений по лимитированному абонементу, проверка доступности мест при записи на занятие, автоматическое списание посещения при входе клиента).

- Common Table Expressions (CTE) и оконные функции – применяются в сложных аналитических представлениях (Views) для ранжирования популярных видов занятий, расчёта скользящего среднего рейтинга тренеров, анализа активности клиентов и выявления «спящих» клиентов.

Для проектирования архитектуры, написания SQL-кода и тестирования используется DataGrip – профессиональная среда разработки от JetBrains, предоставляющая инструменты визуального проектирования баз данных, создания ER-диаграмм, написания и отладки хранимых процедур, анализа планов выполнения запросов и управления версиями SQL-скриптов.

2.2 Функциональные требования к базе данных

Разрабатываемая база данных должна полностью покрывать потребности фитнес-центра:

- Управление клиентами и абонементом: БД должна хранить

информацию о клиентах: полное имя, контактный телефон, электронная почта, дата регистрации в клубе, статус (активен, заблокирован). Необходимо поддерживать различные типы абонементов: лимитированные (фиксированное число посещений), безлимитные (на 1, 3, 6 или 12 месяцев), абонементы с возможностью заморозки (приостановка действия на определённый срок). Для каждого проданного абонемента фиксируется дата начала и окончания действия, количество оплаченных посещений (для лимитированных), сумма покупки, статус (активен, заморожен, истёк). Система должна автоматически контролировать остаток посещений по лимитированному абонементу с запретом записи при исчерпании лимита, а также вести историю посещений клиента с фиксацией даты, времени и типа занятия.

- Управление расписанием, занятиями и тренерами: БД должна поддерживать хранение информации о видах занятий: название (йога, пилатес, кроссфит, единоборства и т.д.), продолжительность в минутах, максимальная вместимость группы. Необходимо реализовать расписание занятий с привязкой к конкретному залу, тренеру, дате и времени начала/окончания. Система должна обеспечивать запись клиентов на занятия с фиксацией даты и времени записи, статуса записи (подтверждена, отменена, проведена), а также автоматическую проверку доступности мест при записи: запрет записи, если количество уже записанных клиентов превышает максимальную вместимость зала. Для тренеров необходимо хранить полное имя, специализацию, контактный телефон, стаж работы, статус (работает, в отпуске, уволен), а также обеспечивать привязку тренеров к конкретным занятиям в расписании. Система должна вести учёт проведённых тренером занятий с фиксацией даты, времени, типа занятия и количества клиентов, а также выполнять расчёт заработной платы тренера на основе проведённых занятий (фиксированная ставка за занятие).

- Управление дополнительными услугами: БД должна хранить каталог дополнительных услуг: название (солярий, массаж, сауна, спортивный бар), цена. Необходимо фиксировать заказы дополнительных услуг клиентами с указанием даты и времени оказания услуги, итоговой стоимости и статуса (забронировано, оказано, отменено), а также обеспечивать привязку заказа к конкретному клиенту.

- Аналитические отчёты и финансовая отчётность: система должна формировать на уровне представлений (Views) следующие отчёты: отчёт по загрузке залов (средняя наполняемость в процентах за выбранный период), отчёт по популярности видов занятий (количество записей, количество проведённых занятий), финансовую сводку (общая выручка, выручка по типам услуг – абонементы / дополнительные услуги, динамика продажи абонементов по месяцам). На основе хранимых данных также должна формироваться статистика по тренерам (количество проведённых занятий, общее количество клиентов на занятиях) и список «спящих» клиентов (абонемент активен, но нет посещений более заданного количества дней).

2.3 Ключевые особенности разрабатываемой базы данных

- Благодаря нормализации до третьей нормальной формы, добавление новых видов занятий, типов абонементов, тренеров или дополнительных услуг производится добавлением записей в справочные таблицы, а не изменением схемы базы данных. Это позволяет легко расширять функциональность без нарушения работы существующих модулей.

- Триггеры автоматически контролируют остаток посещений по лимитированному абонементу при записи на занятие, проверяют доступность мест при добавлении клиента в группу, а также автоматически списывают одно посещение при фиксации входа клиента в фитнес-центр.

- На уровне базы данных реализованы проверки, предотвращающие некорректные операции. Невозможно записать клиента на занятие при отсутствии действующего абонемента или при исчерпании лимита посещений, установить отрицательную сумму платежа за абонемент, изменить статус записи в обход установленной последовательности («подтверждена» → «проведена» / «отменена»).

2.4 Алгоритм обработки данных и методы взаимодействия

При создании новой записи на занятие клиент выбирает вид занятия, дату и время, а также тренера. Система проверяет наличие действующего абонемента у клиента и доступность мест в группе. Если запись подтверждается, она создаётся со статусом «Подтверждена».

При отмене записи клиентом (не позднее установленного времени) статус меняется на «Отменена клиентом». Триггер автоматически освобождает зарезервированное место в группе. Если клиент не отменил запись заранее и не пришёл на занятие, администратор устанавливает статус «Неявка» – в этом случае посещение считается использованным (для лимитированных абонемента) или не возвращается. При фактическом посещении клиент отмечается на стойке администратора или через турникет, статус записи меняется на «Проведена», а триггер автоматически списывает одно посещение (для лимитированных абонемента).

Для формирования аналитических отчётов используются представления, которые агрегируют данные из множества таблиц. Представление финансовой сводки показывает общую выручку, выручку по типам услуг (абонементы / дополнительные услуги) и динамику продажи абонемента по месяцам; представление по загрузке залов – среднюю наполняемость в процентах за выбранный период; представление популярности видов занятий – наиболее востребованные направления с количеством записей и проведённых занятий; представление эффективности тренеров – количество проведённых занятий, общее количество клиентов на занятиях и начисленную заработную плату за период.

3 ИНФОЛОГИЧЕСКАЯ МОДЕЛЬ ПРЕДМЕТНОЙ ОБЛАСТИ

3.1 Модель базы данных

В процессе проектирования в среде DataGrip была разработана даталогическая модель базы данных, включающая 46 взаимосвязанных таблиц. Модель полностью описывает предметную область «фитнес-центр» и покрывает все ключевые бизнес-процессы, выявленные на этапе анализа требований.

В составе модели выделены следующие логические блоки:

- Справочная информация. Данный блок включает таблицы для хранения неизменяемых или редко изменяемых данных: список клиентов, перечень тренеров, каталог видов занятий, типы абонементов, а также перечень дополнительных услуг.

- Управление расписанием и записью на занятия. В эту группу входят таблицы, обеспечивающие ведение расписания занятий с привязкой к конкретному залу, тренеру, дате и времени начала/окончания. Отдельная таблица фиксирует записи клиентов на занятия с указанием даты и времени записи, статуса записи. Для каждой записи предусмотрена автоматическая проверка доступности мест через триггер, сравнивающий количество записанных клиентов с максимальной вместимостью зала.

- Учёт абонементов и посещений. Данный блок содержит таблицы для фиксации продажи абонементов каждому клиенту: дата начала и окончания действия, количество оплаченных посещений (для лимитированных типов), сумма покупки, текущий статус (активен, заморожен, истёк). Таблица истории посещений хранит факты входа клиентов в фитнес-центр с привязкой к конкретному абонементу и занятию. При фиксации входа триггер автоматически списывает одно посещение по лимитированному абонементу, если остаток посещений положителен.

- Финансовые операции. Этот блок отвечает за учёт всех платежей клиентов: сумма, способ оплаты (наличные, карта в клубе, онлайн-перевод), дата и время, привязка к конкретному абонементу или дополнительной услуге.

Схема даталогической модели представлена в приложении.

3.2 Словесное описание модели

В базе данных выделено несколько крупных логических блоков. Всего разработано 46 таблиц.

service – каталог всех услуг (абонементы, доп. услуги). Поля: ser_id, ser_name, ser_category_id, ser_created_date, ser_status, ser_description.

service_category – категории услуг (абонементы / массаж / сауна). Поля: scat_id, scat_name.

price_book – прайс-листы (как в Salesforce). Поля: pboo_id, pboo_name, pboo_active_from, pboo_active_to.

price_book_entry – цена услуги в конкретном прайс-листе. Поля: pent_id, pent_price_book_id, pent_service_id, pent_price.

client – основная сущность клиента. Поля: clie_id, clie_full_name, clie_phone, clie_email, clie_registration_date, clie_status_id.

client_status – справочник статусов клиента. Поля: csta_id, csta_name.

abonement_type – типы абонементов. Поля: atyp_id, atyp_name, atyp_is_limited, atyp_visit_limit, atyp_duration_months, atyp_freeze_allowed.

abonement – купленный абонемент. Поля: abon_id, abon_client_id, abon_type_id, abon_start_date, abon_end_date, abon_remaining_visits, abon_status_id.

abonement_status – статусы абонемента. Поля: astat_id, astat_name.

abonement_freeze – периоды заморозки абонемента. Поля: afre_id, afre_abonement_id, afre_start_date, afre_end_date.

abonement_visit – списания посещений по абонементу. Поля: avis_id, avis_abonement_id, avis_visit_id.

visit – факт входа клиента в клуб. Поля: visi_id, visi_client_id, visi_visit_datetime, visi_visit_type, visi_schedule_id.

visit_type – тип посещения (сам / с тренером). Поля: vtyp_id, vtyp_name.

trainer – тренеры клуба. Поля: trai_id, trai_employee_id, trai_specialization, trai_experience_years, trai_rate.

activity – тип занятия (йога, кроссфит, групповое или с тренером). Поля: acti_id, acti_name, acti_description, acti_is_group_activity, acti_duration_minutes, acti_max_capacity.

hall – залы клуба. Поля: hall_id, hall_name, hall_capacity.

schedule – конкретное занятие в расписании (ядро системы). Поля: sche_id, sche_activity_id, sche_trainer_id, sche_hall_id, sche_start_time, sche_end_time, sche_current_booking_amount.

booking – запись клиента на занятие. Поля: book_id, book_client_id, book_schedule_id, book_abonement_id, book_status_id, book_created_at.

booking_status – статусы записи на занятие. Поля: bsta_id, bsta_name.

service_order – заказ услуги (не абонементной). Поля: sord_id, sord_client_id, sord_service_id, sord_datetime, sord_status_id.

service_order_status – статусы заказа услуг. Поля: sost_id, sost_name.

payment – все платежи клиентов. Поля: paym_id, paym_client_id, paym_total_amount, paym_payment_method_id, paym_created_at.

payment_method – способы оплаты. Поля: pmet_id, pmet_name.

payment_item – детализация платежа (как Salesforce OpportunityLineItem). Поля: pait_id, pait_payment_id, pait_price_book_entry_id, pait_unit_price, pait_discount_id, pait_end_unit_price, pait_quantity, pait_subtotal_amount.

discount – скидки (процентные). Поля: disc_id, disc_description, disc_percent.

client_discount – привязка скидок к клиентам. Поля: cdi_client_id, cdi_discount_id.

equipment_category – категории оборудования (кардио / силовое / инвентарь). Поля: eqca_id, eqca_name, eqca_description.

equipment – единица оборудования. Поля: equi_id, equi_name, equi_category_id, equi_hall_id, equi_purchase_date, equi_status_id, equi_inventory_number.

equipment_status – статус оборудования (active/maintenance/broken/disposed). Поля: eqst_id, eqst_name.

equipment_maintenance – обслуживание оборудования. Поля: eqma_id, eqma_equipment_id, eqma_start_date, eqma_end_date, eqma_description, eqma_cost.

equipment_usage_log – лог использования оборудования клиентами или тренерами. Поля: equs_id, equs_equipment_id, equs_client_id, equs_trainer_id, equs_start_time, equs_end_time.

equipment_supplier – поставщики оборудования. Поля: eqsu_id, eqsu_name, eqsu_phone, eqsu_email.

equipment_purchase – закупки оборудования у поставщиков. Поля: eqpu_id, eqpu_supplier_id, eqpu_purchase_date, eqpu_total_amount.

equipment_purchase_item – состав закупки (какое оборудование, по какой цене, сколько). Поля: epit_id, epit_purchase_id, epit_equipment_id, epit_price, epit_quantity.

employee – сотрудники клуба (администраторы, тренеры, менеджеры, уборщики). Поля: empl_id, empl_full_name, empl_phone, empl_email, empl_hire_date, empl_position_id, empl_status_id, empl_base_salary.

employee_position – должности сотрудников. Поля: epos_id, epos_name.

employee_status – статус сотрудника (active/on_leave/fired). Поля: esta_id, esta_name.

employee_schedule – график работы сотрудников по дням. Поля: esc_id, esc_employee_id, esc_work_date, esc_start_time, esc_end_time.

payroll – начисление зарплаты сотрудникам за период. Поля: payr_id, payr_employee_id, payr_period_start, payr_period_end, payr_total_amount, payr_created_at.

payroll_item – детализация зарплаты (оклад, премия, штраф, оплата занятий). Поля: pait_id, pait_payroll_id, pait_type, pait_amount, pait_description.

trainer_payment – оплата тренеру за проведённое занятие. Поля: trpa_id, trpa_trainer_id, trpa_schedule_id, trpa_amount, trpa_created_at.

vacation – отпуска сотрудников. Поля: vaca_id, vaca_employee_id, vaca_start_date, vaca_end_date, vaca_status.

training_course – курсы повышения квалификации. Поля: trco_id, trco_name, trco_description, trco_duration_hours, trco_cost.

employee_training – прохождение курсов сотрудниками. Поля: emtr_id, emtr_employee_id, emtr_course_id, emtr_start_date, emtr_end_date, emtr_status.

employee_bonus – бонусы сотрудникам. Поля: embo_id, embo_employee_id, embo_amount, embo_reason, embo_created_at.

4 ПОДРОБНОЕ ОПИСАНИЕ БИЗНЕС-ЛОГИКИ

4.1 Хранимые процедуры и функции

Хранимая процедура — это подпрограмма, которая может быть параметризована и предназначена для выполнения множества операций с данными и структурами базы данных. Она сохраняется в самой базе данных и доступна как для вызова из других процедур и триггеров, так и для прямого исполнения.

4.1.1 Вычисление итоговой стоимости платежной позиции

Данная функция предназначена для расчёта стоимости одной позиции в платёжной ведомости с учётом применённой скидки. Функция принимает идентификатор позиции платежа, получает её цену, количество и идентификатор скидки. Если скидка применена, функция вычисляет цену после скидки, умножает на количество и обновляет соответствующие поля в таблице `payment_item`. Это позволяет автоматически поддерживать актуальность данных о стоимости каждой позиции без ручного пересчёта.

```
CREATE FUNCTION CalculatePaymentItemSubtotal(p_payment_item_id INT)
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
    DECLARE v_unit_price DECIMAL(10,2);
    DECLARE v_quantity INT;
    DECLARE v_discount_id INT;
    DECLARE v_discount_percent DECIMAL(5,2);
    DECLARE v_end_unit_price DECIMAL(10,2);
    DECLARE v_subtotal DECIMAL(10,2);

    SELECT pait_unit_price, pait_quantity, pait_discount_id
    INTO v_unit_price, v_quantity, v_discount_id
    FROM payment_item
    WHERE pait_id = p_payment_item_id;

    IF v_discount_id IS NOT NULL THEN
        SELECT disc_percent INTO v_discount_percent
        FROM discount
        WHERE disc_id = v_discount_id;
        SET v_end_unit_price = v_unit_price * (1 - v_discount_percent /
        100);
    ELSE
        SET v_end_unit_price = v_unit_price;
    END IF;

    SET v_subtotal = v_end_unit_price * v_quantity;

    UPDATE payment_item
    SET pait_end_unit_price = v_end_unit_price,
        pait_subtotal_amount = v_subtotal
    WHERE pait_id = p_payment_item_id;

    RETURN v_subtotal;
END;
```

4.1.2 Вычисление общей суммы платежа

Процедура предназначена для пересчёта полной суммы всего платежа на основе всех входящих в него позиций. Она суммирует значения subtotal из всех записей payment_item, связанных с указанным платежом, и обновляет поле paym_total_amount в таблице payment. Процедура гарантирует, что общая сумма платежа всегда соответствует сумме его составных частей. Использование IFNULL исключает ошибки при отсутствии позиций в платеже.

```
CREATE PROCEDURE RecalculatePaymentTotal(IN p_payment_id INT)
BEGIN
    DECLARE v_total DECIMAL(10,2);

    SELECT SUM(pait_subtotal_amount) INTO v_total
    FROM payment_item
    WHERE pait_payment_id = p_payment_id;

    UPDATE payment
    SET paym_total_amount = IFNULL(v_total, 0)
    WHERE paym_id = p_payment_id;
END;
```

4.1.3 Оформление посещения клиента

Процедура регистрирует факт входа клиента в фитнес-центр и списывает соответствующее посещение с его абонеента. Перед оформлением процедура выполняет комплексную проверку: валидирует статус абонеента (активен ли), срок его действия и остаток посещений (для лимитированных абонеентов). Если клиент идёт на конкретное занятие, процедура также проверяет вместимость зала и увеличивает счётчик записей. После успешного прохождения всех проверок создаётся запись в таблице visit, списывается посещение и фиксируется связь между визитом и абонементом. Все операции выполняются в транзакции, что гарантирует целостность данных.

```
CREATE PROCEDURE RegisterVisit(
    IN p_client_id INT,           -- ID клиента
    IN p_visit_type_id INT,       -- Тип посещения (с тренером или без?)
    IN p_schedule_id INT,        -- ID занятия (NULL если клиент пришел
                                -- просто позаниматься)
    IN p_abonement_id INT        -- ID абонеента
)
BEGIN
    DECLARE v_remaining INT;      -- Остаток посещений по абонементу
    DECLARE v_status_id INT       -- Статус абонеента (active / frozen /
                                -- expired)
    DECLARE v_end_date DATE;      -- Дата окончания действия абонеента
    DECLARE v_is_limited INT;
    DECLARE v_max_capacity INT;   -- Максимальная вместимость зала
    DECLARE v_current_bookings INT; -- Текущее количество записей на
                                -- занятие
    DECLARE v_hall_id INT;        -- ID зала, где проходит занятие
    DECLARE v_visit_id INT;       -- ID созданного визита

    START TRANSACTION;
```

```

-- Получаем информацию об абонементе: остаток посещений, статус, дату
  окончания,
-- а также флаг лимитированности (подзапросом в abonement_type)
SELECT abon_remaining_visits, abon_status_id, abon_end_date,
      (SELECT atyp_is_limited FROM abonement_type WHERE atyp_id =
        abon_type_id)
INTO v_remaining, v_status_id, v_end_date, v_is_limited
FROM abonement
WHERE abon_id = p_abonement_id AND abon_client_id = p_client_id;

-- Проверка: абонемент должен быть активен
IF v_status_id != (SELECT astat_id FROM abonement_status WHERE
  astat_name = 'active') THEN
  SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Абонемент не активен';
END IF;

-- Проверка: срок действия не истек
IF v_end_date < CURDATE() THEN
  SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Срок абонемента
                                             истек';
END IF;

-- Проверка: для лимитированного абонемента должен быть хотя бы 1
  визит в остатке
IF v_is_limited = 1 AND v_remaining <= 0 THEN
  SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Лимит посещений
                                             исчерпан';
END IF;

-- Если клиент идет на конкретное занятие (а не просто в зал)
IF p_schedule_id IS NOT NULL THEN
  -- Получаем зал занятия и текущее количество записей
  SELECT sche_hall_id, sche_current_booking_amount INTO v_hall_id,
    v_current_bookings
  FROM schedule WHERE sche_id = p_schedule_id;

  -- Получаем максимальную вместимость зала
  SELECT hall_capacity INTO v_max_capacity
  FROM hall WHERE hall_id = v_hall_id;

  -- Проверка: не превышена ли вместимость зала
  IF v_current_bookings >= v_max_capacity THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Максимальная
      вместимость зала превышена';
  END IF;

  -- Увеличиваем счетчик записей на занятие на 1
  UPDATE schedule SET sche_current_booking_amount =
    sche_current_booking_amount + 1
  WHERE sche_id = p_schedule_id;
END IF;

-- Создаем запись о визите
-- Фиксируем факт входа клиента с текущей датой/временем
INSERT INTO visit (visi_client_id, visi_visit_datetime,
  visi_visit_type, visi_schedule_id)
VALUES (p_client_id, NOW(), p_visit_type_id, p_schedule_id);

SET v_visit_id = LAST_INSERT_ID();

-- Списываем посещение
IF v_is_limited = 1 THEN

```

```

        UPDATE abonement SET abon_remaining_visits =
                                abon_remaining_visits - 1
        WHERE abon_id = p_abonement_id;
END IF;

-- Связываем визит с абонементом
INSERT INTO abonement_visit (avis_abonement_id, avis_visit_id)
VALUES (p_abonement_id, v_visit_id);

COMMIT;
END;

```

4.1.4 Добавление новой услуги фитнес-центра

Процедура автоматизирует процесс добавления новой услуги в систему фитнес-центра. Она одновременно создаёт запись в таблице `service` (с названием, категорией, описанием и статусом "active"), а также добавляет ценовую информацию в прайс-лист. При этом процедура сначала проверяет, существует ли уже активный прайс-лист с указанным именем (без даты окончания), и если да — использует его, если нет — создаёт новый. Это позволяет группировать услуги в разные прайс-листы (например, "Летний 2024", "Сезонный"). После выполнения процедуры новая услуга становится доступной для продажи клиентам.

```

CREATE PROCEDURE AddNewService(
    IN p_service_name VARCHAR(255),
    IN p_category_id INT,
    IN p_description TEXT,
    IN p_price DECIMAL(10,2),
    IN p_price_book_name VARCHAR(255)
)
BEGIN
    DECLARE v_service_id INT;
    DECLARE v_price_book_id INT;

    START TRANSACTION;

    -- Добавляем услугу
    INSERT INTO service (ser_name, ser_category_id, ser_created_date,
                        ser_status, ser_description)
    VALUES (p_service_name, p_category_id, CURDATE(), 'active',
            p_description);

    SET v_service_id = LAST_INSERT_ID();

    -- Проверяем, существует ли уже активный прайс-бук с таким названием
    SELECT pboo_id INTO v_price_book_id
    FROM price_book
    WHERE pboo_name = p_price_book_name
       AND pboo_active_to IS NULL
    LIMIT 1;

    -- Если не существует — создаём новый
    IF v_price_book_id IS NULL THEN
        INSERT INTO price_book (pboo_name, pboo_active_from,
                               pboo_active_to)
        VALUES (p_price_book_name, CURDATE(), NULL);
    END IF;
END;

```

```

        SET v_price_book_id = LAST_INSERT_ID();
    END IF;

    -- Добавляем запись в прайс-бук
    INSERT INTO price_book_entry (pent_price_book_id, pent_service_id,
                                   pent_price)
    VALUES (v_price_book_id, v_service_id, p_price);

    COMMIT;
END;
```

4.1.5 Обновление тренерского опыта работы

Процедура предназначена для планового обновления стажа работы тренера, которое должно запускаться один раз в месяц. Она увеличивает значение поля `trai_experience_years` на 1/12 (одну двенадцатую года), что эквивалентно прибавке одного месяца опыта.

```

CREATE PROCEDURE UpdateTrainerExperience(IN p_trainer_id INT)
BEGIN
    UPDATE trainer
    SET trai_experience_years = trai_experience_years + 1.0/12
    WHERE trai_id = p_trainer_id;
END;
```

4.1.6 Заполнение графика тренера на день

Процедура автоматически создаёт расписание занятий для указанного тренера на определённый период времени. Она принимает идентификатор тренера, зала, типа занятия, стартовое время, количество занятий и длительность каждого занятия. В цикле процедура последовательно вставляет записи в таблицу `schedule`, где каждое следующее занятие начинается сразу после окончания предыдущего.

```

CREATE PROCEDURE GenerateDailySchedule(
    IN p_trainer_id INT,
    IN p_hall_id INT,
    IN p_activity_id INT,
    IN p_start_datetime DATETIME,          -- начало первого занятия
    IN p_number_of_lessons INT,            -- количество занятий подряд
    IN p_lesson_duration_minutes INT       -- длительность одного занятия
                                           в минутах
)
BEGIN
    DECLARE v_counter INT DEFAULT 0;
    DECLARE v_current_start DATETIME;
    DECLARE v_current_end DATETIME;

    SET v_current_start = p_start_datetime;

    WHILE v_counter < p_number_of_lessons DO
        SET v_current_end = DATE_ADD(v_current_start, INTERVAL
                                      p_lesson_duration_minutes MINUTE);

        INSERT INTO schedule (
            sche_activity_id,
            sche_trainer_id,
```



```

        sche_hall_id,
        sche_start_time,
        sche_end_time,
        sche_current_booking_amount
    ) VALUES (
        p_activity_id,
        p_trainer_id,
        p_hall_id,
        v_current_start,
        v_current_end,
        0
    );

    -- Следующее занятие начинается сразу после окончания предыдущего
    SET v_current_start = v_current_end;
    SET v_counter = v_counter + 1;
END WHILE;
END;
```

4.1.7 Вычисление зарплаты тренера

Функция рассчитывает итоговую заработную плату тренера за указанный месяц на основе комплексной формулы. Она учитывает базовый оклад сотрудника (из таблицы employee), количество проведённых групповых и индивидуальных занятий, умноженное на ставку тренера за занятие, а также все начисленные бонусы и наложенные штрафы за указанный период.

```

CREATE FUNCTION CalculateTrainerSalary(
    p_trainer_id INT,
    p_year INT,
    p_month INT
)
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
    DECLARE v_group_count INT DEFAULT 0;
    DECLARE v_individual_count INT DEFAULT 0;
    DECLARE v_rate DECIMAL(10,2);
    DECLARE v_base_salary DECIMAL(10,2);
    DECLARE v_employee_id INT;
    DECLARE v_bonus_sum DECIMAL(10,2) DEFAULT 0;
    DECLARE v_penalty_sum DECIMAL(10,2) DEFAULT 0;
    DECLARE v_period_start DATE;
    DECLARE v_period_end DATE;
    DECLARE v_total DECIMAL(10,2);

    SET v_period_start = DATE(CONCAT(p_year, '-', p_month, '-01'));
    SET v_period_end = LAST_DAY(v_period_start);

    -- Получаем employee_id тренера, его ставку за занятие и базовую
    -- зарплату из employee
    SELECT t.trai_employee_id, t.trai_rate, e.empl_base_salary
    INTO v_employee_id, v_rate, v_base_salary
    FROM trainer t
    JOIN employee e ON t.trai_employee_id = e.empl_id
    WHERE t.trai_id = p_trainer_id;

    -- Считаем групповые занятия
    SELECT COUNT(*) INTO v_group_count
    FROM schedule s
    JOIN activity a ON s.sche_activity_id = a.acti_id
```

```

WHERE s.sche_trainer_id = p_trainer_id
      AND a.acti_is_group_activity = TRUE
      AND s.sche_start_time BETWEEN v_period_start AND v_period_end;

-- Считаем индивидуальные занятия
SELECT COUNT(*) INTO v_individual_count
FROM schedule s
JOIN activity a ON s.sche_activity_id = a.acti_id
WHERE s.sche_trainer_id = p_trainer_id
      AND a.acti_is_group_activity = FALSE
      AND s.sche_start_time BETWEEN v_period_start AND v_period_end;

-- Бонусы
SELECT IFNULL(SUM(embo_amount), 0) INTO v_bonus_sum
FROM employee_bonus
WHERE embo_employee_id = v_employee_id
      AND embo_created_at BETWEEN v_period_start AND v_period_end;

-- Штрафы
SELECT IFNULL(SUM(empe_amount), 0) INTO v_penalty_sum
FROM employee_penalty
WHERE empe_employee_id = v_employee_id
      AND empe_created_at BETWEEN v_period_start AND v_period_end;

-- ИТОГО: базовый оклад + (ставка × количество занятий) + бонусы -
-- штрафы
SET v_total = v_base_salary
              + (v_group_count + v_individual_count) * v_rate
              + v_bonus_sum
              - v_penalty_sum;

RETURN v_total;
END;
```

4.1.8 Вычисление средней загруженности зала за месяц

Функция анализирует загруженность указанного зала за последний месяц и возвращает средний процент заполняемости. Она определяет период как последние 30 дней от текущей даты и собирает данные о вместимости зала и текущем количестве записей на всех занятиях, проходивших в этом зале. Для каждого занятия вычисляется процент заполнения, а затем все эти значения усредняются.

```

CREATE FUNCTION GetAverageHallLoad(p_hall_id INT)
RETURNS DECIMAL(5,2)
DETERMINISTIC
BEGIN
    DECLARE v_capacity INT;
    DECLARE v_avg_load DECIMAL(5,2);
    DECLARE v_period_start DATE;

    SET v_period_start = DATE_SUB(CURDATE(), INTERVAL 1 MONTH);

    SELECT hall_capacity INTO v_capacity
    FROM hall
    WHERE hall_id = p_hall_id;

    SELECT AVG(sche_current_booking_amount / v_capacity * 100) INTO
           v_avg_load
    FROM schedule
```

```

WHERE sche_hall_id = p_hall_id
  AND sche_start_time BETWEEN v_period_start AND CURDATE()
  AND v_capacity > 0;

RETURN IFNULL(v_avg_load, 0);
END;

```

4.1.9 Вычисление среднего количества индивидуальных занятий для тренера в день

Функция предназначена для оценки загрузки конкретного тренера индивидуальными занятиями в течение месяца. Она подсчитывает общее количество индивидуальных занятий (где `acti_is_group_activity = FALSE`), проведённых тренером за указанный месяц, а затем делит это число на количество дней в этом месяце. Результатом является среднее число индивидуальных тренировок, которое тренер проводит в день. Это позволяет оценить равномерность нагрузки тренера и выявить возможные перегрузки или недогрузку. Функция полезна для администраторов при планировании графика работы тренеров и расчёта их премиальной части заработной платы, которая может зависеть от интенсивности индивидуальной работы с клиентами.

```

CREATE FUNCTION GetAvgIndividualLessonsForTrainerPerDay(
  p_trainer_id INT,
  p_year INT,
  p_month INT
)
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
  DECLARE v_individual_count INT DEFAULT 0;
  DECLARE v_days_in_month INT DEFAULT 0;
  DECLARE v_avg DECIMAL(10,2);
  DECLARE v_period_start DATE;
  DECLARE v_period_end DATE;

  SET v_period_start = DATE(CONCAT(p_year, '-', p_month, '-01'));
  SET v_period_end = LAST_DAY(v_period_start);

  -- Считаем количество индивидуальных занятий
  SELECT COUNT(*) INTO v_individual_count
  FROM schedule s
  JOIN activity a ON s.sche_activity_id = a.acti_id
  WHERE s.sche_trainer_id = p_trainer_id
    AND a.acti_is_group_activity = FALSE
    AND s.sche_start_time BETWEEN v_period_start AND v_period_end;

  -- Количество дней в месяце
  SET v_days_in_month = DAY(v_period_end);

  -- Среднее в день
  SET v_avg = v_individual_count / v_days_in_month;

  RETURN IFNULL(v_avg, 0);
END;

```

4.2 Триггеры

Триггер – это объект базы данных, содержащий набор действий, которые автоматически выполняются при возникновении определённого события. Он может срабатывать до, после или вместо операций вставки, обновления или удаления, помогая поддерживать целостность данных и автоматизировать повторяющиеся процессы.

4.2.1 Проверка отмены записи не позднее чем за час до занятия

Триггер проверяет, что отмена записи на занятие происходит не позднее чем за час до его начала. Если попытка отмены совершается позже, операция блокируется с сообщением об ошибке.

```
CREATE TRIGGER check_booking_cancellation_time
BEFORE UPDATE ON booking
FOR EACH ROW
BEGIN
    DECLARE v_schedule_start DATETIME;
    -- Если статус меняется на 'cancelled' (и не был уже cancelled)
    IF NEW.book_status_id = (SELECT bsta_id FROM booking_status WHERE
                           bsta_name = 'cancelled')
        AND OLD.book_status_id != NEW.book_status_id THEN

        -- Получаем время начала занятия
        SELECT sche_start_time INTO v_schedule_start
        FROM schedule
        WHERE sche_id = NEW.book_schedule_id;
        -- Проверяем: отмена возможна только за час и более до начала
        -- занятия
        IF NOW() > DATE_SUB(v_schedule_start, INTERVAL 1 HOUR) THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Отмена записи возможна только за час и
                               более до начала занятия';
        END IF;
    END IF;
END;
```

4.2.2 Продление срока абонеента при создании заморозки

Триггер автоматически продлевает дату окончания абонеента на количество дней заморозки. Это гарантирует, что клиент не теряет оплаченные дни действия абонеента.

```
CREATE TRIGGER extend_abonement_on_freeze
AFTER INSERT ON abonement_freeze
FOR EACH ROW
BEGIN
    DECLARE v_freeze_days INT;

    -- Вычисляем количество дней заморозки
    SET v_freeze_days = DATEDIFF(NEW.afre_end_date,
                                NEW.afre_start_date);

    -- Продлеваем дату окончания абонеента на срок заморозки
```

```

UPDATE abonement
SET abon_end_date = DATE_ADD(abon_end_date, INTERVAL v_freeze_days
                             DAY)
WHERE abon_id = NEW.afre_abonement_id;
END;

```

4.2.3 Автоматическое освобождение места при отмене бронирования

Триггер уменьшает счётчик текущих записей на занятие при отмене бронирования. Это освобождает место для других клиентов, желающих записаться.

```

CREATE TRIGGER free_booking_slot_on_cancellation
AFTER UPDATE ON booking
FOR EACH ROW
BEGIN
    -- Если статус изменился на 'cancelled'
    IF NEW.book_status_id = (SELECT bsta_id FROM booking_status WHERE
                             bsta_name = 'cancelled')
        AND OLD.book_status_id != NEW.book_status_id THEN

        -- Уменьшаем количество записей на занятие
        UPDATE schedule
        SET sche_current_booking_amount=sche_current_booking_amount - 1
        WHERE sche_id = NEW.book_schedule_id
            AND sche_current_booking_amount > 0;
    END IF;
END;

```

4.2.4 Запрет записи клиента на занятие, если у него нет активного абонемент

Триггер проверяет наличие активного абонемента у клиента перед созданием записи на занятие. Если активного абонемента нет, запись блокируется.

```

CREATE TRIGGER check_active_abonement_before_booking
BEFORE INSERT ON booking
FOR EACH ROW
BEGIN
    DECLARE v_has_active_abonement INT;
    -- Проверяем наличие активного абонемента у клиента
    SELECT COUNT(*) INTO v_has_active_abonement
    FROM abonement a
    JOIN abonement_status ast ON a.abon_status_id = ast.astat_id
    WHERE a.abon_client_id = NEW.book_client_id
        AND ast.astat_name = 'active'
        AND a.abon_end_date >= CURDATE();

    -- Если нет активного абонемента
    IF v_has_active_abonement = 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Запись на занятие невозможна: отсутствует
                             активный абонемент';
    END IF;
END;

```

4.2.5 Контроль максимальной вместимости зала при создании бронирования

Триггер проверяет, что количество записанных на занятие клиентов не превышает максимальную вместимость зала. При превышении лимита запись запрещается.

```
CREATE TRIGGER check_capacity_before_booking
BEFORE INSERT ON booking
FOR EACH ROW
BEGIN
    DECLARE v_max_capacity INT;
    DECLARE v_current_bookings INT;
    DECLARE v_activity_id INT;
    DECLARE v_activity_max_capacity INT;

    -- Получаем информацию о занятии и зале
    SELECT s.sche_current_booking_amount, s.sche_activity_id,
           h.hall_capacity
    INTO v_current_bookings, v_activity_id, v_max_capacity
    FROM schedule s
    JOIN hall h ON s.sche_hall_id = h.hall_id
    WHERE s.sche_id = NEW.book_schedule_id;

    -- Получаем максимальную вместимость из типа занятия (если меньше,
    чем у зала)
    SELECT acti_max_capacity INTO v_activity_max_capacity
    FROM activity
    WHERE acti_id = v_activity_id;

    -- Используем минимальную из двух вместимостей
    IF v_activity_max_capacity < v_max_capacity THEN
        SET v_max_capacity = v_activity_max_capacity;
    END IF;

    -- Проверяем, не превышена ли вместимость
    IF v_current_bookings >= v_max_capacity THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Невозможно записаться: группа уже
                             заполнена';
    END IF;
END;
```

4.2.6 Запрет удаления тренера, у которого есть будущие занятия

Триггер запрещает удаление тренера, если у него есть запланированные будущие занятия. Это предотвращает появление занятий в расписании без назначенного тренера.

```
CREATE TRIGGER prevent_trainer_deletion_with_future_schedule
BEFORE DELETE ON trainer
FOR EACH ROW
BEGIN
    DECLARE v_future_lessons INT;

    -- Проверяем, есть ли у тренера будущие занятия в расписании
    SELECT COUNT(*) INTO v_future_lessons
```

```

FROM schedule
WHERE sche_trainer_id = OLD.trai_id
    AND sche_start_time > NOW();

IF v_future_lessons > 0 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Невозможно удалить тренера: у него
                        запланированы будущие занятия';
END IF;
END;
```

4.3 Представления

Представление – это виртуальная таблица, основанная на результате выполнения SQL-запроса. Оно упрощает доступ к данным, скрывая сложные запросы, улучшает читаемость кода и может способствовать повышению производительности.

4.3.1 Отчёт по загрузке залов

Представление показывает среднюю наполняемость каждого зала в процентах за выбранный период. Оно отображает количество фактических записей по сравнению с максимальной вместимостью зала. Данные группируются по дням, что позволяет анализировать пиковые и провальные дни загрузки.

```

CREATE VIEW hall_load_report AS
SELECT
    h.hall_id,
    h.hall_name,
    DATE(s.sche_start_time) AS lesson_date,
    COUNT(b.book_id) AS actual_bookings,
    h.hall_capacity AS max_capacity,
    ROUND(COUNT(b.book_id) / h.hall_capacity * 100, 2) AS load_percent
FROM hall h
JOIN schedule s ON h.hall_id = s.sche_hall_id
LEFT JOIN booking b ON s.sche_id = b.book_schedule_id
    AND b.book_status_id = (SELECT bsta_id FROM booking_status WHERE
bsta_name = 'confirmed')
WHERE s.sche_start_time >= DATE_SUB(CURDATE(), INTERVAL 1 MONTH)
GROUP BY h.hall_id, DATE(s.sche_start_time);
```

4.3.2 Отчёт по популярности видов занятий

Представление отображает количество проведённых занятий и количество записей по каждому виду занятий за месяц. Оно также вычисляет среднюю заполняемость одного занятия, что позволяет оценить популярность каждого направления. Эти данные помогают руководству принимать решения о расширении или сокращении определённых программ.

```
CREATE VIEW lesson_popularity_report AS
SELECT
    a.acti_id,
    a.acti_name,
    COUNT(DISTINCT s.sche_id) AS total_lessons_held,
    COUNT(b.book_id) AS total_bookings,
    ROUND(COUNT(b.book_id) / COUNT(DISTINCT s.sche_id), 1) AS
avg_booking_per_lesson
FROM activity a
JOIN schedule s ON a.acti_id = s.sche_activity_id
LEFT JOIN booking b ON s.sche_id = b.book_schedule_id
    AND b.book_status_id = (SELECT bsta_id FROM booking_status WHERE
bsta_name = 'confirmed')
WHERE s.sche_start_time >= DATE_SUB(CURDATE(), INTERVAL 1 MONTH)
GROUP BY a.acti_id;
```

4.3.3 Финансовая сводка

Представление агрегирует финансовые данные по месяцам, показывая общую выручку, выручку по типам услуг (абонементы и дополнительные услуги) и количество уникальных клиентов. Оно позволяет отслеживать динамику продаж и выявлять сезонные колебания доходов.

```
CREATE VIEW financial_summary AS
SELECT
    DATE_FORMAT(p.paym_created_at, '%Y-%m') AS month,
    SUM(p.paym_total_amount) AS total_revenue,
    SUM(CASE WHEN s.ser_category_id = (SELECT scat_id FROM
service_category WHERE scat_name = 'Абонементы')
    THEN pi.pait_subtotal_amount ELSE 0 END) AS
subscription_revenue,
    SUM(CASE WHEN s.ser_category_id != (SELECT scat_id FROM
service_category WHERE scat_name = 'Абонементы')
    THEN pi.pait_subtotal_amount ELSE 0 END) AS
additional_services_revenue,
    COUNT(DISTINCT p.paym_client_id) AS unique_clients
FROM payment p
JOIN payment_item pi ON p.paym_id = pi.pait_payment_id
JOIN price_book_entry pbe ON pi.pait_price_book_entry_id = pbe.pent_id
JOIN service s ON pbe.pent_service_id = s.ser_id
GROUP BY DATE_FORMAT(p.paym_created_at, '%Y-%m')
ORDER BY month DESC;
```

4.3.4 Отчёт по эффективности тренеров

Представление показывает эффективность работы каждого тренера за последний месяц. Оно отображает количество проведённых занятий, общее

количество записей, среднюю заполняемость и количество уникальных клиентов у каждого тренера.

```
CREATE VIEW trainer_efficiency_report AS
SELECT
    t.trai_id,
    e.empl_full_name AS trainer_name,
    COUNT(DISTINCT s.sche_id) AS total_lessons,
    COUNT(b.book_id) AS total_bookings,
    ROUND(COUNT(b.book_id) / COUNT(DISTINCT s.sche_id), 1) AS
        avg_attendance,
    COUNT(DISTINCT b.book_client_id) AS unique_clients
FROM trainer t
JOIN employee e ON t.trai_employee_id = e.empl_id
JOIN schedule s ON t.trai_id = s.sche_trainer_id
LEFT JOIN booking b ON s.sche_id = b.book_schedule_id
    AND b.book_status_id = (SELECT bsta_id FROM booking_status WHERE
        bsta_name = 'confirmed')
WHERE s.sche_start_time >= DATE_SUB(CURDATE(), INTERVAL 1 MONTH)
GROUP BY t.trai_id;
```

4.3.5 Отчёт по «спящим» клиентам

Представление выявляет клиентов с активными абонементом, которые не посещали фитнес-центр более 30 дней. Оно показывает дату последнего визита и количество пропущенных дней. Эта информация используется маркетинговым отделом для проведения акций по возврату «спящих» клиентов.

```
CREATE VIEW inactive_clients_report AS
SELECT
    c.clie_id,
    c.clie_full_name,
    c.clie_phone,
    c.clie_email,
    a.abon_id,
    a.abon_end_date,
    a.abon_remaining_visits,
    MAX(v.visi_visit_datetime) AS last_visit_date,
    DATEDIFF(CURDATE(), MAX(v.visi_visit_datetime)) AS
        days_since_last_visit
FROM client c
JOIN abonement a ON c.clie_id = a.abon_client_id
JOIN abonement_status ast ON a.abon_status_id = ast.astat_id
LEFT JOIN visit v ON c.clie_id = v.visi_client_id
WHERE ast.astat_name = 'active'
    AND a.abon_end_date > CURDATE()
GROUP BY c.clie_id
HAVING last_visit_date IS NULL OR DATEDIFF(CURDATE(), last_visit_date) > 30
ORDER BY days_since_last_visit DESC;
```

5 ТЕСТИРОВАНИЕ БАЗЫ ДАННЫХ

Тестирование базы данных – ключевой этап разработки, в ходе которого выявляются ошибки, проверяется корректность работы триггеров, хранимых процедур, функций и представлений, а также подтверждается соответствие системы требованиям и ожиданиям пользователей. В таблице 5.1 отображены результаты тестирования бизнес-логики базы данных.

Таблица 5.1 – Тест-кейсы

Действия	Ожидаемый результат	Результат
Вызов функции CalculatePaymentItemSubtotal для payment_item с pait_id=1, где скидка 10%	Функция возвращает сумму со скидкой, поля pait_end_unit_price и pait_subtotal_amount обновлены	Возвращено 900.00, поля обновлены
Вызов процедуры RecalculatePaymentTotal для платежа paym_id=5, состоящего из 3 позиций	Поле paym_total_amount обновлено на сумму всех subtotal позиций	paym_total_amount = 3500.00
Вызов процедуры RegisterVisit с неактивным абонементом (статус 'frozen')	Ошибка с сообщением "Абонемент не активен", транзакция откатена	Ошибка: Абонемент не активен
Вызов процедуры RegisterVisit с лимитированным абонементом, где остаток = 0	Ошибка с сообщением "Лимит посещений исчерпан"	Ошибка: Лимит посещений исчерпан
Вызов процедуры AddNewService с новым названием "Массаж спины" и ценой 1500, прайс-бук "Цены 2024"	Создана новая услуга, создан или найден прайс-бук, добавлена запись в price_book_entry	ser_id = 25, pent_id = 38
Вызов процедуры AddNewService с существующим прайс-буком "Цены 2024"	Услуга добавлена в существующий прайс-бук, новый прайс-бук не создаётся	pent_price_book_id = 2
Вызов процедуры GenerateDailySchedule для тренера с 3 занятиями по 60 минут	В таблицу schedule добавлено 3 записи с последовательным временем	Добавлено 3 записи, время начала каждой следующей = концу предыдущей

Вызов функции CalculateTrainerSalary за январь 2024, где у тренера 10 групповых и 5 индивидуальных занятий, ставка 50, оклад 1000, бонус 200, штраф 50	Возвращено $1000 + (10+5)*50 + 200 - 50 = 1900$	Возвращено 1900.00
Вызов функции GetAverageHallLoad для зала с hall_id=1, где за месяц было 20 занятий с разной заполняемостью	Возвращён средний процент заполняемости за последние 30 дней	Возвращено 65.50%
Попытка отмены записи (UPDATE booking SET book_status_id = статус 'cancelled') за 30 минут до начала занятия	Ошибка с сообщением "Отмена записи возможна только за час и более до начала занятия"	Ошибка: Отмена записи возможна только за час и более
Вставка записи в abonement_freeze на 14 дней для абонеента с end_date = '2024-12-31'	Триггер продлевает abon_end_date на 14 дней, новое значение = '2025-01-14'	abon_end_date = '2025-01-14'
Попытка вставки записи в booking для клиента без активного абонеента	Ошибка с сообщением "Запись на занятие невозможна: отсутствует активный абонемент"	Ошибка: отсутствует активный абонемент
Попытка вставки записи в booking для занятия, где уже записано максимальное количество клиентов (v_current_bookings >= max_capacity)	Ошибка с сообщением "Невозможно записаться: группа уже заполнена"	Ошибка: группа уже заполнена
Попытка удаления тренера с trai_id=5, у которого есть будущие занятия в schedule	Ошибка с сообщением "Невозможно удалить тренера: у него запланированы будущие занятия"	Ошибка: запланированы будущие занятия

В результате выполнения всех тестовых сценариев все хранимые процедуры, функции, триггеры и представления отработали корректно в соответствии с заявленной бизнес-логикой. Ошибки, возникавшие при выполнении тестов с некорректными входными данными, обрабатывались штатно с выдачей информативных сообщений, что подтверждает надёжность реализованных механизмов контроля целостности данных.

ЗАКЛЮЧЕНИЕ

В рамках выполнения курсовой работы была успешно спроектирована и реализована сложная реляционная база данных, предназначенная для автоматизации работы фитнес-центра с расширенной системой учёта абонементов, управления расписанием, контроля посещаемости, бонусной программы и учёта дополнительных услуг. Выполнены все поставленные требования: разработана структура, включающая 46 таблиц, покрывающих широкий спектр бизнес-потребностей от справочной информации (клиенты, тренеры, виды занятий, типы абонементов, дополнительные услуги) до управления расписанием и записью на занятия, учёта проданных абонементов и их статусов, финансовых операций и платежей, а также журнала контроля доступа и служебной информации.

Использование современной СУБД MySQL 8.4 позволило применить продвинутые механизмы обеспечения целостности и производительности данных. Каскадные ограничения внешних ключей гарантируют отсутствие записей без связей и поддерживают ссылочную целостность на всех уровнях базы данных. Значительная часть бизнес-логики была успешно делегирована на уровень базы данных посредством реализации хранимых процедур (оформление посещения клиента, добавление новой услуги, заполнение графика тренера), пользовательских функций (расчёт зарплаты тренера, вычисление загруженности зала, расчёт стоимости платежа) и триггеров (проверка отмены записи за час, продление срока абонемента при заморозке, освобождение места при отмене бронирования, контроль вместимости зала, запрет удаления тренера с будущими занятиями). Разработанные представления обеспечивают удобный доступ к аналитической информации: загрузка залов, популярность видов занятий, финансовая сводка, эффективность тренеров и список неактивных клиентов.

Проведённое тестирование подтвердило корректную работу всех реализованных объектов базы данных. Таким образом, разработанная база данных фитнес-центра готова к опытной эксплуатации и может быть использована в качестве основы для создания полноценного программного продукта по автоматизации работы малых и средних фитнес-клубов.

СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

1. MySQL 8.4 Reference Manual [Электронный ресурс]. – Oracle, 2025. – Режим доступа: <https://dev.mysql.com/doc/refman/8.4/en/>.
2. Куликов, С. С. Работа с MySQL, MSSQL Server и Oracle / С. С. Куликов. – Москва : ДМК Пресс, 2021. – 602 с.
3. Куликов, С. С. Реляционные базы данных в примерах / С. С. Куликов. – Москва : ДМК Пресс, 2021. – 422 с.
4. Бейли, Л. Изучаем SQL / Л. Бейли ; пер. с англ. – Санкт-Петербург : Питер, 2012. – 592 с.
5. Радков, А. В. Основы реляционных баз данных / А. В. Радков. – Москва : Интернет-Университет Информационных Технологий, 2018. – 350 с.
6. Дейт, К. Дж. Введение в системы баз данных / К. Дж. Дейт ; пер. с англ. – 8-е изд. – Москва : Вильямс, 2006. – 1328 с.
7. Тарасов, С. В. MySQL 8 : самоучитель / С. В. Тарасов. – Санкт-Петербург : БХВ-Петербург, 2020. – 432 с.
8. О'Нил, П. Базы данных : принципы и программирование / П. О'Нил, Э. О'Нил ; пер. с англ. – Москва : Мир, 2019. – 736 с.

ПРИЛОЖЕНИЕ А

(обязательное)

```
SET NAMES utf8mb4;
SET FOREIGN_KEY_CHECKS = 0;

-- DROP TABLES

DROP TABLE IF EXISTS employee_penalty;
DROP TABLE IF EXISTS employee_bonus;
DROP TABLE IF EXISTS employee_training;
DROP TABLE IF EXISTS training_course;
DROP TABLE IF EXISTS vacation;
DROP TABLE IF EXISTS trainer_payment;
DROP TABLE IF EXISTS payroll_item;
DROP TABLE IF EXISTS payroll;
DROP TABLE IF EXISTS employee_schedule;
DROP TABLE IF EXISTS employee;
DROP TABLE IF EXISTS employee_status;
DROP TABLE IF EXISTS employee_position;

DROP TABLE IF EXISTS equipment_purchase_item;
DROP TABLE IF EXISTS equipment_purchase;
DROP TABLE IF EXISTS equipment_supplier;
DROP TABLE IF EXISTS equipment_usage_log;
DROP TABLE IF EXISTS equipment_maintenance;
DROP TABLE IF EXISTS equipment;
DROP TABLE IF EXISTS equipment_status;
DROP TABLE IF EXISTS equipment_category;

DROP TABLE IF EXISTS client_discount;
DROP TABLE IF EXISTS discount;

DROP TABLE IF EXISTS payment_item;
DROP TABLE IF EXISTS payment;
DROP TABLE IF EXISTS payment_method;

DROP TABLE IF EXISTS service_order;
DROP TABLE IF EXISTS service_order_status;

DROP TABLE IF EXISTS booking;
DROP TABLE IF EXISTS booking_status;

DROP TABLE IF EXISTS schedule;
DROP TABLE IF EXISTS hall;
DROP TABLE IF EXISTS activity;

DROP TABLE IF EXISTS trainer;

DROP TABLE IF EXISTS visit;
DROP TABLE IF EXISTS visit_type;

DROP TABLE IF EXISTS abonement_visit;
DROP TABLE IF EXISTS abonement_freeze;
DROP TABLE IF EXISTS abonement;
DROP TABLE IF EXISTS abonement_status;
DROP TABLE IF EXISTS abonement_type;

DROP TABLE IF EXISTS client;
DROP TABLE IF EXISTS client_status;
```

```

DROP TABLE IF EXISTS price_book_entry;
DROP TABLE IF EXISTS price_book;

DROP TABLE IF EXISTS service;
DROP TABLE IF EXISTS service_category;

SET FOREIGN_KEY_CHECKS = 1;

CREATE TABLE service_category (
    scat_id INT AUTO_INCREMENT PRIMARY KEY,
    scat_name VARCHAR(100) NOT NULL
) ENGINE=InnoDB;

CREATE TABLE service (
    ser_id INT AUTO_INCREMENT PRIMARY KEY,
    ser_name VARCHAR(255) NOT NULL,
    ser_category_id INT NOT NULL,
    ser_created_date DATETIME NOT NULL,
    ser_status ENUM('active','inactive') NOT NULL,
    ser_description TEXT,
    FOREIGN KEY (ser_category_id) REFERENCES service_category(scat_id)
) ENGINE=InnoDB;

CREATE TABLE price_book (
    pboo_id INT AUTO_INCREMENT PRIMARY KEY,
    pboo_name VARCHAR(255) NOT NULL,
    pboo_active_from DATE NOT NULL,
    pboo_active_to DATE
) ENGINE=InnoDB;

CREATE TABLE price_book_entry (
    pent_id INT AUTO_INCREMENT PRIMARY KEY,
    pent_price_book_id INT NOT NULL,
    pent_service_id INT NOT NULL,
    pent_price DECIMAL(10,2) NOT NULL CHECK (pent_price >= 0),
    FOREIGN KEY (pent_price_book_id) REFERENCES price_book(pboo_id),
    FOREIGN KEY (pent_service_id) REFERENCES service(ser_id)
) ENGINE=InnoDB;

-- CLIENT

CREATE TABLE client_status (
    csta_id INT AUTO_INCREMENT PRIMARY KEY,
    csta_name VARCHAR(50) NOT NULL
);

CREATE TABLE client (
    clie_id INT AUTO_INCREMENT PRIMARY KEY,
    clie_full_name VARCHAR(255) NOT NULL,
    clie_phone VARCHAR(50),
    clie_email VARCHAR(255),
    clie_registration_date DATETIME NOT NULL,
    clie_status_id INT NOT NULL,
    FOREIGN KEY (clie_status_id) REFERENCES client_status(csta_id)
);

-- ABONEMENT

CREATE TABLE abonement_type (
    atyp_id INT AUTO_INCREMENT PRIMARY KEY,
    atyp_name VARCHAR(255) NOT NULL,

```

```

        atyp_is_limited BOOLEAN NOT NULL,
        atyp_visit_limit INT,
        atyp_duration_months INT NOT NULL CHECK (atyp_duration_months > 0),
        atyp_freeze_allowed BOOLEAN NOT NULL
    );

CREATE TABLE abonement_status (
    astat_id INT AUTO_INCREMENT PRIMARY KEY,
    astat_name VARCHAR(50) NOT NULL
);

CREATE TABLE abonement (
    abon_id INT AUTO_INCREMENT PRIMARY KEY,
    abon_client_id INT NOT NULL,
    abon_type_id INT NOT NULL,
    abon_start_date DATE NOT NULL,
    abon_end_date DATE NOT NULL,
    abon_remaining_visits INT NOT NULL CHECK (abon_remaining_visits >=
0),
    abon_status_id INT NOT NULL,
    FOREIGN KEY (abon_client_id) REFERENCES client(clie_id),
    FOREIGN KEY (abon_type_id) REFERENCES abonement_type(atyp_id),
    FOREIGN KEY (abon_status_id) REFERENCES abonement_status(astat_id)
);

CREATE TABLE abonement_freeze (
    afre_id INT AUTO_INCREMENT PRIMARY KEY,
    afre_abonement_id INT NOT NULL,
    afre_start_date DATE NOT NULL,
    afre_end_date DATE NOT NULL,
    FOREIGN KEY (afre_abonement_id) REFERENCES abonement(abon_id)
);

-- VISITS

CREATE TABLE visit_type (
    vtyp_id INT AUTO_INCREMENT PRIMARY KEY,
    vtyp_name VARCHAR(50) NOT NULL
);

CREATE TABLE visit (
    visi_id INT AUTO_INCREMENT PRIMARY KEY,
    visi_client_id INT NOT NULL,
    visi_visit_datetime DATETIME NOT NULL,
    visi_visit_type INT NOT NULL,
    visi_schedule_id INT,
    FOREIGN KEY (visi_client_id) REFERENCES client(clie_id),
    FOREIGN KEY (visi_visit_type) REFERENCES visit_type(vtyp_id)
);

CREATE TABLE abonement_visit (
    avis_id INT AUTO_INCREMENT PRIMARY KEY,
    avis_abonement_id INT NOT NULL,
    avis_visit_id INT NOT NULL,
    FOREIGN KEY (avis_abonement_id) REFERENCES abonement(abon_id),
    FOREIGN KEY (avis_visit_id) REFERENCES visit(visi_id)
);

-- TRAINER / EMPLOYEE

CREATE TABLE employee_position (
    epos_id INT AUTO_INCREMENT PRIMARY KEY,

```



```

        epos_name VARCHAR(100) NOT NULL
    );

CREATE TABLE employee_status (
    esta_id INT AUTO_INCREMENT PRIMARY KEY,
    esta_name VARCHAR(50) NOT NULL
);

CREATE TABLE employee (
    empl_id INT AUTO_INCREMENT PRIMARY KEY,
    empl_full_name VARCHAR(255) NOT NULL,
    empl_phone VARCHAR(50),
    empl_email VARCHAR(255),
    empl_hire_date DATE NOT NULL,
    empl_position_id INT NOT NULL,
    empl_status_id INT NOT NULL,
    empl_base_salary DECIMAL(10,2),
    FOREIGN KEY (empl_position_id) REFERENCES
employee_position(epos_id),
    FOREIGN KEY (empl_status_id) REFERENCES employee_status(esta_id)
);

CREATE TABLE trainer (
    trai_id INT AUTO_INCREMENT PRIMARY KEY,
    trai_employee_id INT NOT NULL,
    trai_specialization VARCHAR(255),
    trai_experience_years INT,
    trai_rate DECIMAL(10,2),
    FOREIGN KEY (trai_employee_id) REFERENCES employee(empl_id)
);

-- ACTIVITY / SCHEDULE

CREATE TABLE activity (
    acti_id INT AUTO_INCREMENT PRIMARY KEY,
    acti_name VARCHAR(255) NOT NULL,
    acti_description TEXT,
    acti_is_group_activity BOOLEAN NOT NULL,
    acti_duration_minutes INT NOT NULL,
    acti_max_capacity INT
);

CREATE TABLE hall (
    hall_id INT AUTO_INCREMENT PRIMARY KEY,
    hall_name VARCHAR(100) NOT NULL,
    hall_capacity INT NOT NULL
);

CREATE TABLE schedule (
    sche_id INT AUTO_INCREMENT PRIMARY KEY,
    sche_activity_id INT NOT NULL,
    sche_trainer_id INT NOT NULL,
    sche_hall_id INT NOT NULL,
    sche_start_time DATETIME NOT NULL,
    sche_end_time DATETIME NOT NULL,
    sche_current_booking_amount INT NOT NULL DEFAULT 0,
    FOREIGN KEY (sche_activity_id) REFERENCES activity(acti_id),
    FOREIGN KEY (sche_trainer_id) REFERENCES trainer(trai_id),
    FOREIGN KEY (sche_hall_id) REFERENCES hall(hall_id)
);

ALTER TABLE visit

```

```

ADD FOREIGN KEY (visi_schedule_id) REFERENCES schedule(sche_id);

-- BOOKING

CREATE TABLE booking_status (
    bsta_id INT AUTO_INCREMENT PRIMARY KEY,
    bsta_name VARCHAR(50) NOT NULL
);

CREATE TABLE booking (
    book_id INT AUTO_INCREMENT PRIMARY KEY,
    book_client_id INT NOT NULL,
    book_schedule_id INT NOT NULL,
    book_abonement_id INT NOT NULL,
    book_status_id INT NOT NULL,
    book_created_at DATETIME NOT NULL,
    FOREIGN KEY (book_client_id) REFERENCES client(clie_id),
    FOREIGN KEY (book_schedule_id) REFERENCES schedule(sche_id),
    FOREIGN KEY (book_abonement_id) REFERENCES abonement(abon_id),
    FOREIGN KEY (book_status_id) REFERENCES booking_status(bsta_id)
);

-- PAYMENTS

CREATE TABLE payment_method (
    pmet_id INT AUTO_INCREMENT PRIMARY KEY,
    pmet_name VARCHAR(50) NOT NULL
);

CREATE TABLE payment (
    paym_id INT AUTO_INCREMENT PRIMARY KEY,
    paym_client_id INT NOT NULL,
    paym_total_amount DECIMAL(10,2) NOT NULL CHECK (paym_total_amount >=
0),
    paym_payment_method_id INT NOT NULL,
    paym_created_at DATETIME NOT NULL,
    FOREIGN KEY (paym_client_id) REFERENCES client(clie_id),
    FOREIGN KEY (paym_payment_method_id) REFERENCES
payment_method(pmet_id)
);

CREATE TABLE discount (
    disc_id INT AUTO_INCREMENT PRIMARY KEY,
    disc_description TEXT,
    disc_percent DECIMAL(5,2) NOT NULL CHECK (disc_percent >= 0 AND
disc_percent <= 100)
);

CREATE TABLE payment_item (
    pait_id INT AUTO_INCREMENT PRIMARY KEY,
    pait_payment_id INT NOT NULL,
    pait_price_book_entry_id INT NOT NULL,
    pait_unit_price DECIMAL(10,2) NOT NULL,
    pait_discount_id INT,
    pait_end_unit_price DECIMAL(10,2),
    pait_quantity INT NOT NULL CHECK (pait_quantity > 0),
    pait_subtotal_amount DECIMAL(10,2) NOT NULL,
    FOREIGN KEY (pait_payment_id) REFERENCES payment(paym_id),
    FOREIGN KEY (pait_price_book_entry_id) REFERENCES
price_book_entry(pent_id),
    FOREIGN KEY (pait_discount_id) REFERENCES discount(disc_id)
);

```

```

CREATE TABLE client_discount (
    cdi_client_id INT,
    cdi_discount_id INT,
    PRIMARY KEY (cdi_client_id, cdi_discount_id),
    FOREIGN KEY (cdi_client_id) REFERENCES client(clie_id),
    FOREIGN KEY (cdi_discount_id) REFERENCES discount(disc_id)
);

-- EQUIPMENT

CREATE TABLE equipment_category (
    eqca_id INT AUTO_INCREMENT PRIMARY KEY,
    eqca_name VARCHAR(100) NOT NULL,
    eqca_description TEXT
);

CREATE TABLE equipment_status (
    eqst_id INT AUTO_INCREMENT PRIMARY KEY,
    eqst_name VARCHAR(50) NOT NULL
);

CREATE TABLE equipment (
    equi_id INT AUTO_INCREMENT PRIMARY KEY,
    equi_name VARCHAR(255) NOT NULL,
    equi_category_id INT NOT NULL,
    equi_hall_id INT,
    equi_purchase_date DATE,
    equi_status_id INT NOT NULL,
    equi_inventory_number VARCHAR(100) NOT NULL UNIQUE,
    FOREIGN KEY (equi_category_id) REFERENCES
equipment_category(eqca_id),
    FOREIGN KEY (equi_hall_id) REFERENCES hall(hall_id),
    FOREIGN KEY (equi_status_id) REFERENCES equipment_status(eqst_id)
);

CREATE TABLE equipment_maintenance (
    eqma_id INT AUTO_INCREMENT PRIMARY KEY,
    eqma_equipment_id INT NOT NULL,
    eqma_start_date DATE NOT NULL,
    eqma_end_date DATE,
    eqma_description TEXT,
    eqma_cost DECIMAL(10,2),
    FOREIGN KEY (eqma_equipment_id) REFERENCES equipment(equi_id)
);

CREATE TABLE equipment_usage_log (
    equs_id INT AUTO_INCREMENT PRIMARY KEY,
    equs_equipment_id INT NOT NULL,
    equs_client_id INT,
    equs_trainer_id INT,
    equs_start_time DATETIME NOT NULL,
    equs_end_time DATETIME,
    FOREIGN KEY (equs_equipment_id) REFERENCES equipment(equi_id),
    FOREIGN KEY (equs_client_id) REFERENCES client(clie_id),
    FOREIGN KEY (equs_trainer_id) REFERENCES trainer(trai_id)
);

-- HR

CREATE TABLE employee_schedule (
    esc_id INT AUTO_INCREMENT PRIMARY KEY,

```

```

        esc_employee_id INT NOT NULL,
        esc_work_date DATE NOT NULL,
        esc_start_time TIME NOT NULL,
        esc_end_time TIME NOT NULL,
        FOREIGN KEY (esc_employee_id) REFERENCES employee(empl_id)
    );

CREATE TABLE payroll (
    payr_id INT AUTO_INCREMENT PRIMARY KEY,
    payr_employee_id INT NOT NULL,
    payr_period_start DATE NOT NULL,
    payr_period_end DATE NOT NULL,
    payr_total_amount DECIMAL(10,2) NOT NULL,
    payr_created_at DATETIME NOT NULL,
    FOREIGN KEY (payr_employee_id) REFERENCES employee(empl_id)
);

CREATE TABLE payroll_item (
    pait_id INT AUTO_INCREMENT PRIMARY KEY,
    pait_payroll_id INT NOT NULL,
    pait_type VARCHAR(50) NOT NULL,
    pait_amount DECIMAL(10,2) NOT NULL,
    pait_description TEXT,
    FOREIGN KEY (pait_payroll_id) REFERENCES payroll(payr_id)
);

CREATE TABLE trainer_payment (
    trpa_id INT AUTO_INCREMENT PRIMARY KEY,
    trpa_trainer_id INT NOT NULL,
    trpa_schedule_id INT NOT NULL,
    trpa_amount DECIMAL(10,2) NOT NULL,
    trpa_created_at DATETIME NOT NULL,
    FOREIGN KEY (trpa_trainer_id) REFERENCES trainer(trai_id),
    FOREIGN KEY (trpa_schedule_id) REFERENCES schedule(sche_id)
);

CREATE TABLE vacation (
    vaca_id INT AUTO_INCREMENT PRIMARY KEY,
    vaca_employee_id INT NOT NULL,
    vaca_start_date DATE NOT NULL,
    vaca_end_date DATE NOT NULL,
    vaca_status VARCHAR(50) NOT NULL,
    FOREIGN KEY (vaca_employee_id) REFERENCES employee(empl_id)
);

CREATE TABLE training_course (
    trco_id INT AUTO_INCREMENT PRIMARY KEY,
    trco_name VARCHAR(255) NOT NULL,
    trco_description TEXT,
    trco_duration_hours INT NOT NULL,
    trco_cost DECIMAL(10,2)
);

CREATE TABLE employee_training (
    emtr_id INT AUTO_INCREMENT PRIMARY KEY,
    emtr_employee_id INT NOT NULL,
    emtr_course_id INT NOT NULL,
    emtr_start_date DATE NOT NULL,
    emtr_end_date DATE,
    emtr_status VARCHAR(50) NOT NULL,
    FOREIGN KEY (emtr_employee_id) REFERENCES employee(empl_id),
    FOREIGN KEY (emtr_course_id) REFERENCES training_course(trco_id)
);

```

```

);

CREATE TABLE employee_bonus (
    embo_id INT AUTO_INCREMENT PRIMARY KEY,
    embo_employee_id INT NOT NULL,
    embo_amount DECIMAL(10,2) NOT NULL,
    embo_reason TEXT,
    embo_created_at DATETIME NOT NULL,
    FOREIGN KEY (embo_employee_id) REFERENCES employee(empl_id)
);

CREATE TABLE employee_penalty (
    empe_id INT AUTO_INCREMENT PRIMARY KEY,
    empe_employee_id INT NOT NULL,
    empe_amount DECIMAL(10,2) NOT NULL,
    empe_reason TEXT,
    empe_created_at DATETIME NOT NULL,
    FOREIGN KEY (empe_employee_id) REFERENCES employee(empl_id)
);

CREATE TABLE service_order_status (
    sost_id INT AUTO_INCREMENT PRIMARY KEY,
    sost_name VARCHAR(50) NOT NULL
) ENGINE=InnoDB;

CREATE TABLE service_order (
    sord_id INT AUTO_INCREMENT PRIMARY KEY,
    sord_client_id INT NOT NULL,
    sord_service_id INT NOT NULL,
    sord_datetime DATETIME NOT NULL,
    sord_status_id INT NOT NULL,
    FOREIGN KEY (sord_client_id) REFERENCES client(clie_id),
    FOREIGN KEY (sord_service_id) REFERENCES service(ser_id),
    FOREIGN KEY (sord_status_id) REFERENCES
service_order_status(sost_id)
) ENGINE=InnoDB;

CREATE TABLE equipment_supplier (
    eqsu_id INT AUTO_INCREMENT PRIMARY KEY,
    eqsu_name VARCHAR(255) NOT NULL,
    eqsu_phone VARCHAR(50),
    eqsu_email VARCHAR(255)
) ENGINE=InnoDB;

CREATE TABLE equipment_purchase (
    eqpu_id INT AUTO_INCREMENT PRIMARY KEY,
    eqpu_supplier_id INT NOT NULL,
    eqpu_purchase_date DATE NOT NULL,
    eqpu_total_amount DECIMAL(10,2) NOT NULL CHECK (eqpu_total_amount >=
0),
    FOREIGN KEY (eqpu_supplier_id) REFERENCES
equipment_supplier(eqsu_id)
) ENGINE=InnoDB;

CREATE TABLE equipment_purchase_item (
    epit_id INT AUTO_INCREMENT PRIMARY KEY,
    epit_purchase_id INT NOT NULL,
    epit_equipment_id INT NOT NULL,
    epit_price DECIMAL(10,2) NOT NULL CHECK (epit_price >= 0),

```

```

        epit_quantity INT NOT NULL CHECK (epit_quantity > 0),
        FOREIGN KEY (epit_purchase_id) REFERENCES
equipment_purchase(eppu_id),
        FOREIGN KEY (epit_equipment_id) REFERENCES equipment(equi_id)
    ) ENGINE=InnoDB;

```

Хранимые процедуры и функции

```

CREATE FUNCTION CalculatePaymentItemSubtotal(p_payment_item_id INT)
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
    DECLARE v_unit_price DECIMAL(10,2);
    DECLARE v_quantity INT;
    DECLARE v_discount_id INT;
    DECLARE v_discount_percent DECIMAL(5,2);
    DECLARE v_end_unit_price DECIMAL(10,2);
    DECLARE v_subtotal DECIMAL(10,2);

    SELECT pait_unit_price, pait_quantity, pait_discount_id
    INTO v_unit_price, v_quantity, v_discount_id
    FROM payment_item
    WHERE pait_id = p_payment_item_id;

    IF v_discount_id IS NOT NULL THEN
        SELECT disc_percent INTO v_discount_percent
        FROM discount
        WHERE disc_id = v_discount_id;
        SET v_end_unit_price = v_unit_price * (1 - v_discount_percent /
        100);
    ELSE
        SET v_end_unit_price = v_unit_price;
    END IF;

    SET v_subtotal = v_end_unit_price * v_quantity;

    UPDATE payment_item
    SET pait_end_unit_price = v_end_unit_price,
        pait_subtotal_amount = v_subtotal
    WHERE pait_id = p_payment_item_id;

    RETURN v_subtotal;
END;

CREATE PROCEDURE RecalculatePaymentTotal(IN p_payment_id INT)
BEGIN
    DECLARE v_total DECIMAL(10,2);

    SELECT SUM(pait_subtotal_amount) INTO v_total
    FROM payment_item
    WHERE pait_payment_id = p_payment_id;

    UPDATE payment
    SET paym_total_amount = IFNULL(v_total, 0)
    WHERE paym_id = p_payment_id;
END;

CREATE PROCEDURE RegisterVisit(
    IN p_client_id INT,          -- ID клиента
    IN p_visit_type_id INT,      -- Тип посещения (с тренером или без?)
    IN p_schedule_id INT,        -- ID занятия (NULL если клиент пришел
                                -- просто позаниматься)

```

```

        IN p_abonement_id INT          -- ID абонементa
    )
BEGIN
    DECLARE v_remaining INT;           -- Остаток посещений по абонементу
    DECLARE v_status_id INT            -- Статус абонементa (active / frozen /
                                        expired)

    DECLARE v_end_date DATE;           -- Дата окончания действия абонементa
    DECLARE v_is_limited INT;
    DECLARE v_max_capacity INT;        -- Максимальная вместимость зала
    DECLARE v_current_bookings INT;    -- Текущее количество записей на
                                        занятие

    DECLARE v_hall_id INT;             -- ID зала, где проходит занятие
    DECLARE v_visit_id INT;            -- ID созданного визита

    START TRANSACTION;

    -- Получаем информацию об абонементе: остаток посещений, статус,
дату
        окончания,
    -- а также флаг лимитированности (подзапросом в abonement_type)
    SELECT abon_remaining_visits, abon_status_id, abon_end_date,
        (SELECT atyp_is_limited FROM abonement_type WHERE atyp_id =
            abon_type_id)
    INTO v_remaining, v_status_id, v_end_date, v_is_limited
    FROM abonement
    WHERE abon_id = p_abonement_id AND abon_client_id = p_client_id;

    -- Проверка: абонемент должен быть активен
    IF v_status_id != (SELECT astat_id FROM abonement_status WHERE
                        astat_name = 'active') THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Абонемент не
активен';
    END IF;

    -- Проверка: срок действия не истек
    IF v_end_date < CURDATE() THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Срок абонементa
истек';
    END IF;

    -- Проверка: для лимитированного абонементa должен быть хотя бы 1
        визит в остатке
    IF v_is_limited = 1 AND v_remaining <= 0 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Лимит посещений
исчерпан';
    END IF;

    -- Если клиент идет на конкретное занятие (а не просто в зал)
    IF p_schedule_id IS NOT NULL THEN
        -- Получаем зал занятия и текущее количество записей
        SELECT sche_hall_id, sche_current_booking_amount INTO v_hall_id,
            v_current_bookings
        FROM schedule WHERE sche_id = p_schedule_id;

        -- Получаем максимальную вместимость зала
        SELECT hall_capacity INTO v_max_capacity
        FROM hall WHERE hall_id = v_hall_id;

        -- Проверка: не превышена ли вместимость зала
        IF v_current_bookings >= v_max_capacity THEN
            SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Максимальная
вместимость зала превышена';
        
```

```

        END IF;

        -- Увеличиваем счетчик записей на занятие на 1
        UPDATE schedule SET sche_current_booking_amount =
            sche_current_booking_amount + 1
        WHERE sche_id = p_schedule_id;
    END IF;

    -- Создаем запись о визите
    -- Фиксируем факт входа клиента с текущей датой/временем
    INSERT INTO visit (visi_client_id, visi_visit_datetime,
        visi_visit_type, visi_schedule_id)
    VALUES (p_client_id, NOW(), p_visit_type_id, p_schedule_id);

    SET v_visit_id = LAST_INSERT_ID();

    -- Списываем посещение
    IF v_is_limited = 1 THEN
        UPDATE abonement SET abon_remaining_visits =
            abon_remaining_visits - 1
        WHERE abon_id = p_abonement_id;
    END IF;

    -- Связываем визит с абонементом
    INSERT INTO abonement_visit (avis_abonement_id, avis_visit_id)
    VALUES (p_abonement_id, v_visit_id);

    COMMIT;
END;

CREATE PROCEDURE AddNewService(
    IN p_service_name VARCHAR(255),
    IN p_category_id INT,
    IN p_description TEXT,
    IN p_price DECIMAL(10,2),
    IN p_price_book_name VARCHAR(255)
)
BEGIN
    DECLARE v_service_id INT;
    DECLARE v_price_book_id INT;

    START TRANSACTION;

    -- Добавляем услугу
    INSERT INTO service (ser_name, ser_category_id, ser_created_date,
        ser_status, ser_description)
    VALUES (p_service_name, p_category_id, CURDATE(), 'active',
        p_description);

    SET v_service_id = LAST_INSERT_ID();

    -- Проверяем, существует ли уже активный прайс-бук с таким названием
    SELECT pboo_id INTO v_price_book_id
    FROM price_book
    WHERE pboo_name = p_price_book_name
        AND pboo_active_to IS NULL
    LIMIT 1;

    -- Если не существует — создаём новый
    IF v_price_book_id IS NULL THEN
        INSERT INTO price_book (pboo_name, pboo_active_from,
            pboo_active_to)

```



```

VALUES (p_price_book_name, CURDATE(), NULL);

SET v_price_book_id = LAST_INSERT_ID();
END IF;

-- Добавляем запись в прайс-бук
INSERT INTO price_book_entry (pent_price_book_id, pent_service_id,
                             pent_price)
VALUES (v_price_book_id, v_service_id, p_price);

COMMIT;
END;

CREATE PROCEDURE UpdateTrainerExperience(IN p_trainer_id INT)
BEGIN
    UPDATE trainer
    SET trai_experience_years = trai_experience_years + 1.0/12
    WHERE trai_id = p_trainer_id;
END;

CREATE PROCEDURE GenerateDailySchedule(
    IN p_trainer_id INT,
    IN p_hall_id INT,
    IN p_activity_id INT,
    IN p_start_datetime DATETIME,          -- начало первого занятия
    IN p_number_of_lessons INT,           -- количество занятий подряд
    IN p_lesson_duration_minutes INT      -- длительность одного занятия
                                         в минутах
)
BEGIN
    DECLARE v_counter INT DEFAULT 0;
    DECLARE v_current_start DATETIME;
    DECLARE v_current_end DATETIME;

    SET v_current_start = p_start_datetime;

    WHILE v_counter < p_number_of_lessons DO
        SET v_current_end = DATE_ADD(v_current_start, INTERVAL
                                     p_lesson_duration_minutes MINUTE);

        INSERT INTO schedule (
            sche_activity_id,
            sche_trainer_id,
            sche_hall_id,
            sche_start_time,
            sche_end_time,
            sche_current_booking_amount
        ) VALUES (
            p_activity_id,
            p_trainer_id,
            p_hall_id,
            v_current_start,
            v_current_end,
            0
        );

        -- Следующее занятие начинается сразу после окончания
предыдущего
        SET v_current_start = v_current_end;
        SET v_counter = v_counter + 1;
    END WHILE;
END;

```

```

CREATE FUNCTION CalculateTrainerSalary(
    p_trainer_id INT,
    p_year INT,
    p_month INT
)
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
    DECLARE v_group_count INT DEFAULT 0;
    DECLARE v_individual_count INT DEFAULT 0;
    DECLARE v_rate DECIMAL(10,2);
    DECLARE v_base_salary DECIMAL(10,2);
    DECLARE v_employee_id INT;
    DECLARE v_bonus_sum DECIMAL(10,2) DEFAULT 0;
    DECLARE v_penalty_sum DECIMAL(10,2) DEFAULT 0;
    DECLARE v_period_start DATE;
    DECLARE v_period_end DATE;
    DECLARE v_total DECIMAL(10,2);

    SET v_period_start = DATE(CONCAT(p_year, '-', p_month, '-01'));
    SET v_period_end = LAST_DAY(v_period_start);

    -- Получаем employee_id тренера, его ставку за занятие и базовую
    -- зарплату из employee
    SELECT t.trai_employee_id, t.trai_rate, e.empl_base_salary
    INTO v_employee_id, v_rate, v_base_salary
    FROM trainer t
    JOIN employee e ON t.trai_employee_id = e.empl_id
    WHERE t.trai_id = p_trainer_id;

    -- Считаем групповые занятия
    SELECT COUNT(*) INTO v_group_count
    FROM schedule s
    JOIN activity a ON s.sche_activity_id = a.acti_id
    WHERE s.sche_trainer_id = p_trainer_id
        AND a.acti_is_group_activity = TRUE
        AND s.sche_start_time BETWEEN v_period_start AND v_period_end;

    -- Считаем индивидуальные занятия
    SELECT COUNT(*) INTO v_individual_count
    FROM schedule s
    JOIN activity a ON s.sche_activity_id = a.acti_id
    WHERE s.sche_trainer_id = p_trainer_id
        AND a.acti_is_group_activity = FALSE
        AND s.sche_start_time BETWEEN v_period_start AND v_period_end;

    -- Бонусы
    SELECT IFNULL(SUM(embo_amount), 0) INTO v_bonus_sum
    FROM employee_bonus
    WHERE embo_employee_id = v_employee_id
        AND embo_created_at BETWEEN v_period_start AND v_period_end;

    -- Штрафы
    SELECT IFNULL(SUM(empe_amount), 0) INTO v_penalty_sum
    FROM employee_penalty
    WHERE empe_employee_id = v_employee_id
        AND empe_created_at BETWEEN v_period_start AND v_period_end;

    -- ИТОГО: базовый оклад + (ставка × количество занятий) + бонусы -
    -- штрафы
    SET v_total = v_base_salary

```

```

        + (v_group_count + v_individual_count) * v_rate
        + v_bonus_sum
        - v_penalty_sum;

    RETURN v_total;
END;

CREATE FUNCTION GetAverageHallLoad(p_hall_id INT)
RETURNS DECIMAL(5,2)
DETERMINISTIC
BEGIN
    DECLARE v_capacity INT;
    DECLARE v_avg_load DECIMAL(5,2);
    DECLARE v_period_start DATE;

    SET v_period_start = DATE_SUB(CURDATE(), INTERVAL 1 MONTH);

    SELECT hall_capacity INTO v_capacity
    FROM hall
    WHERE hall_id = p_hall_id;

    SELECT AVG(sche_current_booking_amount / v_capacity * 100) INTO
        v_avg_load
    FROM schedule
    WHERE sche_hall_id = p_hall_id
        AND sche_start_time BETWEEN v_period_start AND CURDATE()
        AND v_capacity > 0;

    RETURN IFNULL(v_avg_load, 0);
END;

CREATE FUNCTION GetAvgIndividualLessonsForTrainerPerDay(
    p_trainer_id INT,
    p_year INT,
    p_month INT
)
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
    DECLARE v_individual_count INT DEFAULT 0;
    DECLARE v_days_in_month INT DEFAULT 0;
    DECLARE v_avg DECIMAL(10,2);
    DECLARE v_period_start DATE;
    DECLARE v_period_end DATE;

    SET v_period_start = DATE(CONCAT(p_year, '-', p_month, '-01'));
    SET v_period_end = LAST_DAY(v_period_start);

    -- Считаем количество индивидуальных занятий
    SELECT COUNT(*) INTO v_individual_count
    FROM schedule s
    JOIN activity a ON s.sche_activity_id = a.acti_id
    WHERE s.sche_trainer_id = p_trainer_id
        AND a.acti_is_group_activity = FALSE
        AND s.sche_start_time BETWEEN v_period_start AND v_period_end;

    -- Количество дней в месяце
    SET v_days_in_month = DAY(v_period_end);

    -- Среднее в день
    SET v_avg = v_individual_count / v_days_in_month;
    RETURN IFNULL(v_avg, 0);
END;

```

Триггеры

```
CREATE TRIGGER check_booking_cancellation_time
BEFORE UPDATE ON booking
FOR EACH ROW
BEGIN
    DECLARE v_schedule_start DATETIME;
    -- Если статус меняется на 'cancelled' (и не был уже cancelled)
    IF NEW.book_status_id = (SELECT bsta_id FROM booking_status WHERE
                             bsta_name = 'cancelled')
        AND OLD.book_status_id != NEW.book_status_id THEN

        -- Получаем время начала занятия
        SELECT sche_start_time INTO v_schedule_start
        FROM schedule
        WHERE sche_id = NEW.book_schedule_id;
        -- Проверяем: отмена возможна только за час и более до начала
        занятия
        IF NOW() > DATE_SUB(v_schedule_start, INTERVAL 1 HOUR) THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Отмена записи возможна только за час и
                               более до начала занятия';
        END IF;
    END IF;
END;

CREATE TRIGGER extend_abonement_on_freeze
AFTER INSERT ON abonement_freeze
FOR EACH ROW
BEGIN
    DECLARE v_freeze_days INT;

    -- Вычисляем количество дней заморозки
    SET v_freeze_days = DATEDIFF(NEW.afre_end_date,
                                NEW.afre_start_date);

    -- Продлеваем дату окончания абонемента на срок заморозки
    UPDATE abonement
    SET abon_end_date = DATE_ADD(abon_end_date, INTERVAL v_freeze_days
                                DAY)
    WHERE abon_id = NEW.afre_abonement_id;
END;

CREATE TRIGGER free_booking_slot_on_cancellation
AFTER UPDATE ON booking
FOR EACH ROW
BEGIN
    -- Если статус изменился на 'cancelled'
    IF NEW.book_status_id = (SELECT bsta_id FROM booking_status WHERE
                             bsta_name = 'cancelled')
        AND OLD.book_status_id != NEW.book_status_id THEN

        -- Уменьшаем количество записей на занятие
        UPDATE schedule
        SET sche_current_booking_amount=sche_current_booking_amount - 1
        WHERE sche_id = NEW.book_schedule_id
        AND sche_current_booking_amount > 0;
    END IF;
END;

CREATE TRIGGER check_active_abonement_before_booking
BEFORE INSERT ON booking
```

```

FOR EACH ROW
BEGIN
    DECLARE v_has_active_abonement INT;
    -- Проверяем наличие активного абонемента у клиента
    SELECT COUNT(*) INTO v_has_active_abonement
    FROM abonement a
    JOIN abonement_status ast ON a.abon_status_id = ast.astat_id
    WHERE a.abon_client_id = NEW.book_client_id
    AND ast.astat_name = 'active'
    AND a.abon_end_date >= CURDATE();

    -- Если нет активного абонемента
    IF v_has_active_abonement = 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Запись на занятие невозможна: отсутствует
                                активный абонемент';
    END IF;
END;

CREATE TRIGGER check_capacity_before_booking
BEFORE INSERT ON booking
FOR EACH ROW
BEGIN
    DECLARE v_max_capacity INT;
    DECLARE v_current_bookings INT;
    DECLARE v_activity_id INT;
    DECLARE v_activity_max_capacity INT;

    -- Получаем информацию о занятии и зале
    SELECT s.sche_current_booking_amount, s.sche_activity_id,
           h.hall_capacity
    INTO v_current_bookings, v_activity_id, v_max_capacity
    FROM schedule s
    JOIN hall h ON s.sche_hall_id = h.hall_id
    WHERE s.sche_id = NEW.book_schedule_id;

    -- Получаем максимальную вместимость из типа занятия (если меньше,
    чем у зала)
    SELECT acti_max_capacity INTO v_activity_max_capacity
    FROM activity
    WHERE acti_id = v_activity_id;

    -- Используем минимальную из двух вместимостей
    IF v_activity_max_capacity < v_max_capacity THEN
        SET v_max_capacity = v_activity_max_capacity;
    END IF;

    -- Проверяем, не превышена ли вместимость
    IF v_current_bookings >= v_max_capacity THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Невозможно записаться: группа уже
                                заполнена';
    END IF;
END;

CREATE TRIGGER prevent_trainer_deletion_with_future_schedule
BEFORE DELETE ON trainer
FOR EACH ROW
BEGIN
    DECLARE v_future_lessons INT;

    -- Проверяем, есть ли у тренера будущие занятия в расписании

```

```

SELECT COUNT(*) INTO v_future_lessons
FROM schedule
WHERE sche_trainer_id = OLD.trai_id
AND sche_start_time > NOW();

IF v_future_lessons > 0 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Невозможно удалить тренера: у него
                        запланированы будущие занятия';
END IF;
END;
```

Представления

```

CREATE VIEW hall_load_report AS
SELECT
    h.hall_id,
    h.hall_name,
    DATE(s.sche_start_time) AS lesson_date,
    COUNT(b.book_id) AS actual_bookings,
    h.hall_capacity AS max_capacity,
    ROUND(COUNT(b.book_id) / h.hall_capacity * 100, 2) AS load_percent
FROM hall h
JOIN schedule s ON h.hall_id = s.sche_hall_id
LEFT JOIN booking b ON s.sche_id = b.book_schedule_id
AND b.book_status_id = (SELECT bsta_id FROM booking_status WHERE
bsta_name = 'confirmed')
WHERE s.sche_start_time >= DATE_SUB(CURDATE(), INTERVAL 1 MONTH)
GROUP BY h.hall_id, DATE(s.sche_start_time);

CREATE VIEW lesson_popularity_report AS
SELECT
    a.acti_id,
    a.acti_name,
    COUNT(DISTINCT s.sche_id) AS total_lessons_held,
    COUNT(b.book_id) AS total_bookings,
    ROUND(COUNT(b.book_id) / COUNT(DISTINCT s.sche_id), 1) AS
avg_booking_per_lesson
FROM activity a
JOIN schedule s ON a.acti_id = s.sche_activity_id
LEFT JOIN booking b ON s.sche_id = b.book_schedule_id
AND b.book_status_id = (SELECT bsta_id FROM booking_status WHERE
bsta_name = 'confirmed')
WHERE s.sche_start_time >= DATE_SUB(CURDATE(), INTERVAL 1 MONTH)
GROUP BY a.acti_id;

CREATE VIEW financial_summary AS
SELECT
    DATE_FORMAT(p.paym_created_at, '%Y-%m') AS month,
    SUM(p.paym_total_amount) AS total_revenue,
    SUM(CASE WHEN s.ser_category_id = (SELECT scat_id FROM
        service_category WHERE scat_name = 'Абонементы')
    THEN pi.pait_subtotal_amount ELSE 0 END) AS
        subscription_revenue,
    SUM(CASE WHEN s.ser_category_id != (SELECT scat_id FROM
        service_category WHERE scat_name = 'Абонементы')
    THEN pi.pait_subtotal_amount ELSE 0 END) AS
        additional_services_revenue,
    COUNT(DISTINCT p.paym_client_id) AS unique_clients
FROM payment p
JOIN payment_item pi ON p.paym_id = pi.pait_payment_id
JOIN price_book_entry pbe ON pi.pait_price_book_entry_id = pbe.pent_id
```

```

JOIN service s ON pbe.pent_service_id = s.ser_id
GROUP BY DATE_FORMAT(p.paym_created_at, '%Y-%m')
ORDER BY month DESC;

```

```

CREATE VIEW trainer_efficiency_report AS
SELECT
    t.trai_id,
    e.empl_full_name AS trainer_name,
    COUNT(DISTINCT s.sche_id) AS total_lessons,
    COUNT(b.book_id) AS total_bookings,
    ROUND(COUNT(b.book_id) / COUNT(DISTINCT s.sche_id), 1) AS
        avg_attendance,
    COUNT(DISTINCT b.book_client_id) AS unique_clients
FROM trainer t
JOIN employee e ON t.trai_employee_id = e.empl_id
JOIN schedule s ON t.trai_id = s.sche_trainer_id
LEFT JOIN booking b ON s.sche_id = b.book_schedule_id
    AND b.book_status_id = (SELECT bsta_id FROM booking_status WHERE
        bsta_name = 'confirmed')
WHERE s.sche_start_time >= DATE_SUB(CURDATE(), INTERVAL 1 MONTH)
GROUP BY t.trai_id;

```

```

CREATE VIEW inactive_clients_report AS
SELECT
    c.clie_id,
    c.clie_full_name,
    c.clie_phone,
    c.clie_email,
    a.abon_id,
    a.abon_end_date,
    a.abon_remaining_visits,
    MAX(v.visi_visit_datetime) AS last_visit_date,
    DATEDIFF(CURDATE(), MAX(v.visi_visit_datetime)) AS
        days_since_last_visit
FROM client c
JOIN abonement a ON c.clie_id = a.abon_client_id
JOIN abonement_status ast ON a.abon_status_id = ast.astat_id
LEFT JOIN visit v ON c.clie_id = v.visi_client_id
WHERE ast.astat_name = 'active'
    AND a.abon_end_date > CURDATE()
GROUP BY c.clie_id
HAVING last_visit_date IS NULL OR DATEDIFF(CURDATE(), last_visit_date) > 30
ORDER BY days_since_last_visit DESC;

```